HTTP & REST APIS

CHEAT SHEETS

A Language-Agnostic Reference for Building and Calling Web APIs

HTTP/1.1–3 · Methods · Status Codes · Headers · Caching · Cookies · Sessions · JWT · OAuth 2.0 + PKCE · OpenID Connect · CORS · REST Design · Pagination · Versioning · Webhooks · OpenAPI · GraphQL

30 REFERENCE SHEETS

CONTENTS

GETTING STARTED

01 How HTTP Works
02 Anatomy of a Request & Response
03 Testing APIs with curl & Tools

METHODS

04 Reading: GET, HEAD & OPTIONS
05 Writing: POST, PUT, PATCH & DELETE
06 Safe, Idempotent & Cacheable Methods

STATUS CODES

07 1xx Informational & 2xx Success
08 3xx Redirects & Not Modified
09 4xx Client Errors
10 5xx Server Errors & Error Bodies

HEADERS & COOKIES

11 Core Headers & Content Negotiation
12 Caching Headers
13 Cookies & Sessions

AUTHENTICATION & CORS

14 Authentication Basics
15 JSON Web Tokens (JWT)
16 OAuth 2.0 Concepts
17 OAuth Flows: Authorization Code + PKCE & OpenID Connect
18 CORS

REST API DESIGN

19 REST Principles
20 Resource & URL Design
21 Pagination, Filtering & Sorting
22 Versioning & Evolving APIs
23 Rate Limits, Async Jobs & Webhooks
24 Documenting with OpenAPI

GRAPHQL

25 GraphQL Basics & GraphQL vs REST
26 GraphQL Schema & Types (SDL)
27 GraphQL Queries, Mutations & Variables
28 GraphQL Resolvers, Errors & Performance

QUICK REFERENCE

29 Quick Reference — Methods & Status Codes
30 Quick Reference — Headers, Auth & CORS

Covers RFC 9110 · OAuth 2.0 + PKCE · OpenAPI 3.1 · GraphQL · Express, Django, Spring, Rails, ASP.NET & Go (2026)

Begin → Sheet 01

HTTP

Table of Contents

30 sheets · syntax · examples · notes

GETTING STARTED 01 – 03

- 01 How HTTP Works
- 02 Anatomy of a Request …
- 03 Testing APIs with curl …

METHODS 04 – 06

- 04 Reading: GET, HEAD & …
- 05 Writing: POST, PUT, PAT…
- 06 Safe, Idempotent & Ca…

STATUS CODES 07 – 10

- 07 1xx Informational & 2xx…
- 08 3xx Redirects & Not M…
- 09 4xx Client Errors
- 10 5xx Server Errors & Err…

HEADERS & COOKIES 11 – 13

- 11 Core Headers & Conte…
- 12 Caching Headers
- 13 Cookies & Sessions

AUTHENTICATION & CORS 14 – 18

- 14 Authentication Basics
- 15 JSON Web Tokens (JW…
- 16 OAuth 2.0 Concepts
- 17 OAuth Flows: Authoriz…
- 18 CORS

REST API DESIGN 19 – 24

- 19 REST Principles
- 20 Resource & URL Design
- 21 Pagination, Filtering & …
- 22 Versioning & Evolving …
- 23 Rate Limits, Async Job…
- 24 Documenting with Op…

GRAPHQL 25 – 28

- 25 GraphQL Basics & Gra…
- 26 GraphQL Schema & Ty…
- 27 GraphQL Queries, Mut…
- 28 GraphQL Resolvers, Err…

QUICK REFERENCE 29 – 30

- 29 Quick Reference — Me…
- 30 Quick Reference — He…

Sheet 01

How HTTP Works

client / server · request / response · stateless · TCP + TLS · HTTP/1.1 vs 2 vs 3 · HTTPS · HSTS

REQUESTS IN, RESPONSES OUT

HTTP is a text-based **request / response** protocol: a client (browser, app, `curl`, another server) sends a request, and the server sends back exactly one response. It is **stateless**: each request carries everything the server needs (auth, cookies, IDs), so any server behind a load balancer can answer it. **HTTPS** is the same protocol inside a **TLS** tunnel. HTTP/1.1, HTTP/2 and HTTP/3 change how bytes travel, not what methods, status codes and headers mean (RFC 9110).

NOTES

- Default ports: `80` for `http://`, `443` for `https://`
- State lives in cookies, tokens or the database, never in the connection
- HTTP/2 and HTTP/3 are binary on the wire, but tools show them as HTTP/1.1-style text
- Browsers only use HTTP/2 and HTTP/3 over TLS

SECURITY CHECK

- Plain `http://` sends tokens and cookies in clear text: serve every API over HTTPS
- Add `Strict-Transport-Security` so browsers never try `http://` again
- Redirect 80 → 443 for pages, but reject API calls on port 80 instead of redirecting them

ONE ROUND TRIP

```
GET /api/users/42 HTTP/1.1
Host: api.example.com
Accept: application/json
User-Agent: curl/8.10.1

HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 37
Cache-Control: no-store
Strict-Transport-Security: max-age=31536000; includeSubDomains

{"id":42,"name":"Ada","role":"admin"}
```

CODE EXPLAINED

→ Line 1 is the **request line**: method · path · version

→ `Host` is required in HTTP/1.1; one IP can serve many sites

→ Response starts with the **status line**: version · code · reason

→ A blank line separates headers from the body in both directions

→ `Content-Length` is the body size in bytes (37 here)

IN YOUR STACK

- **Express** — `app.get('/api/users/:id', (req, res) => res.json(user))`
- **Django** — `path('api/users/<int:id>/', views.user)`, view returns `JsonResponse`
- **Spring** — `@GetMapping("/api/users/{id}")` on a `@RestController`
- **Rails** — `get "/api/users/:id", to: "users#show"` in `config/routes.rb`
- **ASP.NET** — `app.MapGet("/api/users/{id}", (int id) => ...)`
- **Go** — `mux.HandleFunc("GET /api/users/{id}", h)` (Go 1.22+ patterns)

HTTP VERSIONS

VERSION	TRANSPORT	WHAT CHANGED
`HTTP/1.1`	TCP (+TLS)	Text; one request at a time per connection; keep-alive
`HTTP/2`	TCP + TLS	Binary frames; many streams on one connection; header compression
`HTTP/3`	QUIC (UDP)	No TCP head-of-line blocking; faster handshakes; survives network changes

WHO SITS IN THE MIDDLE

HOP	WHAT IT DOES TO YOUR REQUEST
`CDN`	Caches responses near users; may serve without hitting you
`Load balancer`	Picks a server; often ends TLS
`Reverse proxy`	nginx / Caddy; adds `X-Forwarded-For` / `-Proto`
`API gateway`	Auth, rate limits, routing to services
`App server`	Your Express / Django / Spring / Rails code

⚠ COMMON MISTAKES

- Keeping per-user state in server memory: the next request may land on a different server
- Trusting the client IP from the socket behind a proxy; read `X-Forwarded-For` only from trusted proxies
- Assuming HTTP/2 changes your API: methods, codes and headers mean the same thing

✓ TIPS & BEST PRACTICES

- See the raw exchange any time with `curl -v` (sheet 03)
- Behind a proxy, configure your framework's trusted-proxy setting so `https` and client IP are right
- Turn on HTTP/2 at the proxy or CDN; app servers rarely need to change

QUICK SUMMARY

- `request` — method + path + headers + optional body
- `response` — status + headers + optional body
- `stateless` — every request stands alone
- `HTTPS` — HTTP over TLS; required for anything real

Anatomy of a Request & Response

scheme · host · port · path · query · fragment · percent-encoding · request line · status line · body

EVERY PART HAS A JOB

A **URL** names the resource: scheme, host, optional port, **path**, optional **query string** and a **fragment** the browser keeps to itself. The request puts the method and path on the **request line**, then **headers** (one `Name: value` per line, names case-insensitive), a blank line, and an optional **body**. The response mirrors that shape with a **status line** instead.

NOTES

- Path identifies the resource; query narrows or shapes it (filters, page)
- The `#fragment` is never sent to the server
- Header names are case-insensitive: `content-type` = `Content-Type`
- HTTP/2+ sends headers lowercase and the request line as `:method`, `:path` pseudo-headers

REQUEST AND RESPONSE, PART BY PART

```
POST /api/v1/orders?notify=true HTTP/1.1
Host: shop.example.com
Content-Type: application/json
Content-Length: 28
Authorization: Bearer eyJhbGciOiJSUzI1NiJ9...

{"sku":"A-100","quantity":2}

HTTP/1.1 201 Created
Location: /api/v1/orders/9001
Content-Type: application/json; charset=utf-8

{"id":9001,"status":"pending"}
```

CODE EXPLAINED

- → `POST` method · `/api/v1/orders` path · `?notify=true` query
- → `Content-Type` says how to parse the body; `Content-Length` its byte size
- → `Authorization` carries credentials (sheet 14)
- → `201 Created` + `Location` points to the new resource
- → `charset=utf-8` is a media-type parameter after the `;`

IN YOUR STACK

- **Express** — `req.method` · `req.path` · `req.query.notify` · `req.get('Host')` · `req.body` (needs `express.json()`)
- **Django** — `request.method` · `request.path` · `request.GET['notify']` · `request.headers['Host']` · `request.body`
- **Spring** — `@PathVariable` · `@RequestParam` · `@RequestHeader` · `@RequestBody`
- **Rails** — `request.method` · `request.path` · `params[:notify]` · `request.headers["Host"]` · `request.raw_post`
- **ASP.NET** — `[FromRoute]` · `[FromQuery]` · `[FromHeader]` · `[FromBody]`, or `HttpContext.Request`
- **Go** — `r.Method` · `r.URL.Path` · `r.URL.Query().Get("notify")` · `r.Host` · `r.Body`

DEBUG IT

- DevTools → Network → click the request → **Headers** shows the request and status lines; **Payload** shows query and body
- Server sees an empty body? Check `Content-Type` matches the parser you registered
- A `+` in a query arrives as a space in most parsers: encode it as `%2B`

URL PARTS: HTTPS://ADA@SHOP.EXAMPLE.COM:8443/API/ORDERS?PAGE=2#TOP

PART	VALUE	NOTES
scheme	https	Picks protocol + default port
userinfo	ada@	Deprecated; never put passwords here
host	shop.example.com	Sent as the Host header
port	8443	Omitted when default (443)
path	/api/orders	Case-sensitive on most servers
query	page=2	key=value pairs joined by &
fragment	top	Browser only; not sent

PERCENT-ENCODING

CHAR	ENCODED	WHY
space	%20	+ only in form bodies / queries
/	%2F	Inside a single path segment
? & = #	%3F %26 %3D %23	Inside a query value
+	%2B	Or it decodes as a space
é	%C3%A9	UTF-8 bytes, each encoded

⚠ COMMON MISTAKES

- Building URLs with string concatenation: unencoded `&` or `#` in a value breaks the query
- Putting secrets or tokens in the query string: they end up in logs, history and `Referer`
- Expecting the server to see `#fragment` values

✓ TIPS & BEST PRACTICES

- Use your language's URL builder (`URLSearchParams`, `urllib.parse.urlencode`, `UriComponentsBuilder`)
- Repeat keys for lists: `?tag=a&tag=b`; document it, since frameworks differ
- Keep URLs under ~2,000 characters; long filters belong in a POST body

QUICK SUMMARY

- `URL` — scheme://host:port/path?query#fragment
- `request line` — METHOD /path HTTP/version
- `headers` — metadata; blank line; then `body`
- `%XX` — percent-encode reserved characters

Sheet 03

Testing APIs with curl & Tools

curl -X / -H / -d / -i / -v · --json · -u · -L · HTTPie · Postman / Bruno · DevTools · copy as cURL

TALK TO AN API BY HAND

`curl` is on every Mac, Linux box and Windows 10+, and it is the shared language of API docs: every example on these sheets can be pasted into a terminal. –v prints the whole conversation, which is the fastest way to see what really went over the wire. GUI clients (Postman, Bruno, Insomnia) save requests as collections, and the browser's **Network** tab shows what your front end actually sent.

HOUSE STYLE FOR THIS SET

- In `curl -v` output, `>` lines are sent, `<` lines are received, `*` lines are curl's own notes
- Raw messages show request, blank line, response; `Host` is often left out to save space
- `...` marks a trimmed token or body
- `$TOKEN`, `$KEY` are shell variables you set first

CURL YOU'LL USE EVERY DAY

```
API=https://api.example.com
curl $API/users/42                    # GET, prints body
curl -i $API/users/42                 # + status line & headers
curl -I $API/users/42                 # HEAD: headers only
curl -v $API/users/42                 # full > request / < response
curl -X DELETE $API/users/42          # choose the method
curl -H "Accept: application/json" \
     -H "Authorization: Bearer $TOKEN" $API/me
curl -d 'name=Ada&role=admin' $API/users # form POST
curl --json '{"name":"Ada"}' $API/users # JSON POST
curl -u ada:s3cret $API/private       # Basic auth
curl -F avatar=@me.png $API/upload    # multipart
curl -L https://example.com/old       # follow redirects
curl -sS -o /dev/null -w '%{http_code}\n' $API/health
```

CODE EXPLAINED

→ `-d` switches to POST and sends `application/x-www-form-urlencoded`

→ `--json` (curl 7.82+) = POST + JSON `Content-Type` and `Accept`

→ `-X` only changes the method word; it doesn't add a body

→ `-F` builds `multipart/form-data`; `@file` uploads a file

→ `-w '%{http_code}'` prints just the status, handy in scripts

IN YOUR STACK

- **Express** — `supertest`: `await request(app).get('/users/42').expect(200)`
- **Django** — `self.client.get('/users/42/')` in `TestCase`; DRF adds `APIClient`
- **Spring** — `MockMvc`: `mvc.perform(get("/users/42")).andExpect(status().isOk())`
- **Rails** — Integration test: `get user_path(42)` then `assert_response :success`
- **ASP.NET** — `WebApplicationFactory<Program>` → `client.GetAsync("/users/42")`
- **Go** — `httptest.NewRecorder()` + `h.ServeHTTP(rec, req)`, or `httptest.NewServer`

MORE CURL FLAGS

FLAG	DOES
`-s / -S`	Silent / still show errors
`-o file / -O`	Save body to a file
`-k`	Skip TLS checks (local dev only)
`--http2 / --http3`	Force a protocol version
`-b / -c file`	Send / save cookies
`--retry 3`	Retry on transient errors
`-H 'X: '`	Remove a default header

OTHER TOOLS

TOOL	GOOD FOR
`HTTPie`	`http POST :3000/users name=Ada` — JSON by default
`Postman`	Collections, environments, team sharing
`Bruno`	Collections as plain files in git
`.http files`	VS Code REST Client / JetBrains HTTP Client
`DevTools Network`	Right-click → Copy → **Copy as cURL**

⚠ COMMON MISTAKES

- Using `-X POST` with `-d`: redundant, and `-X` with `-L` re-sends the forced method after a redirect
- Forgetting quotes around JSON: the shell eats the braces and quotes
- Leaving `-k` in scripts that reach production

✓ TIPS & BEST PRACTICES

- Reproduce a browser bug exactly: DevTools → **Copy as cURL**, then trim headers
- Put a token in a variable once: `export TOKEN=...`
- Pipe JSON to `jq` for readable output: `curl -s ... | jq .`

QUICK SUMMARY

- `-v` — see the whole exchange
- `-i` / `-I` — headers with body / headers only
- `--json` · `-d` · `-F` — JSON · form · multipart
- `-H` · `-u` · `-L` — header · Basic auth · follow redirects

Reading: GET, HEAD & OPTIONS

METHODS THAT ONLY READ

GET fetches a representation of a resource; everything that shapes the result (filters, paging, sort) goes in the **query string**, not a body. **HEAD** is GET without the body, so you can check size, type or freshness cheaply. **OPTIONS** asks what a resource supports; servers answer with an `Allow` header, and browsers use it for **CORS preflight** (sheet 18). All three are **safe**: they must not change server state.

NOTES

- A GET body has no defined meaning; proxies and caches may drop it
- HEAD must return the same headers GET would, minus the body
- `OPTIONS *` asks about the whole server; rarely used
- Conditional GET (`If-None-Match`) can return `304` with no body (sheet 12)

THREE READ REQUESTS

```
GET /products?category=books&sort=-price&page=2 HTTP/1.1
Host: api.example.com
Accept: application/json

HEAD /files/report.pdf HTTP/1.1
Host: api.example.com

HTTP/1.1 200 OK
Content-Type: application/pdf
Content-Length: 482113
Last-Modified: Tue, 27 Oct 2026 14:02:11 GMT

OPTIONS /products/7 HTTP/1.1
Host: api.example.com

HTTP/1.1 204 No Content
Allow: GET, HEAD, PUT, PATCH, DELETE, OPTIONS
```

CODE EXPLAINED

→ Query carries the filter, sort and page; the path stays `/products`

→ HEAD returns the PDF's size and date without downloading it

→ `Content-Length` on a HEAD response is the size GET would send

→ OPTIONS answers with `Allow` ; `204` because there's no body

IN YOUR STACK

- **Express** — `app.get()` also answers HEAD automatically; OPTIONS via `app.options()` or `cors()`
- **Django** — Class-based views answer HEAD with `get()` and OPTIONS automatically; `@require_GET` for functions
- **Spring** — `@GetMapping` handles HEAD and OPTIONS (with `Allow`) out of the box
- **Rails** — `get` routes also match HEAD; OPTIONS needs a route or `rack-cors`
- **ASP.NET** — `MapGet()` ; add HEAD with `MapMethods(path, ["HEAD"], ...)` if clients need it
- **Go** — A `"GET /path"` pattern also matches HEAD

DEBUG IT

- `curl -I url` sends a real HEAD; compare its headers with `curl -i`
- A search that "ignores" its filters may be sending them in a GET body: move them to the query
- `405` on OPTIONS from a browser usually means CORS isn't set up (sheet 18)

THE READ METHODS

METHOD	REQUEST BODY	RESPONSE BODY	TYPICAL SUCCESS
`GET`	No	Yes	`200` · `304` · `206`
`HEAD`	No	No (headers only)	`200` · `304`
`OPTIONS`	Rare	Optional	`204` + `Allow`

COMPLEX SEARCHES

OPTION	WHEN
`GET + query`	Default; bookmarkable and cacheable
`POST /search`	Filters too long or structured for a URL
`QUERY method`	IETF draft: safe method with a body; not widely supported yet

⚠ COMMON MISTAKES

- Changing data in a GET handler (e.g. `GET /users/42/delete`): crawlers and prefetch will trigger it
- Sending a JSON body with GET: it works locally, then a proxy or client library drops it
- Returning a body on HEAD, or different headers than GET

✓ TIPS & BEST PRACTICES

- Use HEAD for "does it exist?" and "has it changed?" checks on large files
- Keep query names consistent across endpoints: `page` , `limit` , `sort` , `fields`
- Let the framework answer HEAD/OPTIONS; override only when you must

QUICK SUMMARY

- `GET` — read; params in the query
- `HEAD` — GET without the body
- `OPTIONS` — what's allowed; CORS preflight
- `safe` — none of them change state

Sheet 05

Writing: POST, PUT, PATCH & DELETE

POST create · PUT replace · PATCH partial · JSON Merge Patch · JSON Patch · DELETE · 201 + Location

METHODS THAT CHANGE THINGS

POST asks the server to process the body, most often to **create** a resource whose URL the server picks. **PUT replaces** the resource at a URL the client knows with the full body. **PATCH** applies a **partial** change, described by a patch format: **JSON Merge Patch** (send the fields that change, `null` removes) or **JSON Patch** (a list of operations). **DELETE** removes the resource.

NOTES

- PUT with missing fields means "clear them"; that's why PATCH exists
- PUT can also create when the client chooses the ID: `PUT /users/ada`
- Merge Patch can't set a value to `null` or edit arrays item by item; JSON Patch can
- DELETE usually returns `204`, or `200` with the deleted item

SECURITY CHECK

- **Mass assignment**: never bind the whole body to the model; allow-list fields so clients can't set `role` or `is_admin`
- Check ownership on every write: `PATCH /orders/9001` must belong to the caller (BOLA, OWASP API #1)

CREATE, REPLACE, PATCH, DELETE

```
POST /users HTTP/1.1
Content-Type: application/json

{"name":"Ada","email":"ada@example.com"}

HTTP/1.1 201 Created
Location: /users/42

PUT /users/42 HTTP/1.1
Content-Type: application/json

{"name":"Ada L.","email":"ada@example.com","role":"editor"}

PATCH /users/42 HTTP/1.1
Content-Type: application/merge-patch+json

{"name":"Ada Lovelace","role":null}

PATCH /users/42 HTTP/1.1
Content-Type: application/json-patch+json

[{"op":"replace","path":"/name","value":"Ada"},
 {"op":"remove","path":"/role"}]

DELETE /users/42 HTTP/1.1

HTTP/1.1 204 No Content
```

CODE EXPLAINED

→ POST: server picks ID 42 and returns it in `Location`

→ PUT: the whole user, every field, at a known URL

→ Merge Patch (RFC 7396): changed fields only; `null` deletes `role`

→ JSON Patch (RFC 6902): ordered ops, applied all-or-nothing

→ `Host` lines left out to save space

IN YOUR STACK

- **Express** — `app.post/put/patch/delete()`; `res.status(201).location(url).json(user)`
- **Django** — DRF `ModelViewSet` → `create` · `update` · `partial_update` · `destroy`
- **Spring** — `@PostMapping` · `@PutMapping` · `@PatchMapping` · `@DeleteMapping`; `ResponseEntity.created(uri)`
- **Rails** — `resources :users` → `create` · `update` (PUT and PATCH) · `destroy`; `params.require(:user).permit(:name)`
- **ASP.NET** — `MapPost` · `MapPut` · `MapPatch` · `MapDelete`; `Results.Created($"/users/{id}", user)`
- **Go** — `"POST /users"`, `"PATCH /users/{id}"` patterns; `w.WriteHeader(http.StatusCreated)`

DEBUG IT

- `415 Unsupported Media Type`: the `Content-Type` doesn't match what the endpoint parses
- Handler sees an empty body: no JSON body parser registered (`express.json()`) or wrong `Content-Type`
- `405 Method Not Allowed` + `Allow` header: the route exists, just not for this method

WRITE METHODS COMPARED

METHOD	BODY	SUCCESS	IDEMPOTENT
POST	New item / command	201 + Location · 200 · 202	✖
PUT	Full representation	200 · 204 · 201	✔
PATCH	Patch document	200 · 204	✖ (can be)
DELETE	None	204 · 200 · 202	✔

JSON PATCH OPERATIONS

OP	EXAMPLE
add	`{"op":"add","path":"/tags/-","value":"vip"}`
remove	`{"op":"remove","path":"/role"}`
replace	`{"op":"replace","path":"/name","value":"Ada"}`
move / copy	`{"op":"move","from":"/a","path":"/b"}`
test	Fail the whole patch unless a value matches

⚠ COMMON MISTAKES

- Using PUT for partial updates: fields the client left out get wiped
- Returning `200` with an empty body after create instead of `201` + `Location`
- Treating DELETE of an already-deleted item as a `500`; return `404` or `204`

✓ TIPS & BEST PRACTICES

- Accept `application/merge-patch+json` for PATCH; it's the simplest format for clients
- Return the updated resource from PUT/PATCH so clients don't need a follow-up GET
- Soft-delete (`deleted_at`) when you need undo or audit history

QUICK SUMMARY

- `POST` — create / process; `201` + `Location`
- `PUT` — replace whole resource
- `PATCH` — partial update via a patch format
- `DELETE` — remove; `204`

Safe, Idempotent & Cacheable Methods

WHICH REQUESTS ARE SAFE TO REPEAT

A **safe** method doesn't change state (GET, HEAD, OPTIONS). An **idempotent** method has the same effect whether it runs once or ten times (all safe methods plus PUT and DELETE). This matters because networks fail: a client that times out can't know if the server acted, and only idempotent requests are safe to **retry blindly**. For POST, an `Idempotency-Key` header lets the server spot a retry and replay the first result.

NOTES

- Idempotent is about server **state**, not the response: a second DELETE may return `404`
- `Idempotency-Key` is an IETF draft, used by Stripe, Adyen and others
- Cacheable by default: GET and HEAD; POST only with explicit freshness headers
- Browsers and proxies may retry idempotent requests on their own

RETRYING A PAYMENT SAFELY

```
KEY=$(uuidgen)                          # one key per logical operation

curl --json '{"amount":2500,"currency":"usd"}' \
     -H "Idempotency-Key: $KEY" https://api.example.com/payments
# 201 Created ... but the client timed out and never saw it

curl --json '{"amount":2500,"currency":"usd"}' \
     -H "Idempotency-Key: $KEY" https://api.example.com/payments
# 201 Created, same body: server replayed the stored result

curl --json '{"amount":9999,"currency":"usd"}' \
     -H "Idempotency-Key: $KEY" https://api.example.com/payments
# 422: same key, different payload is rejected

# PUT is idempotent by design: two calls, one final state
curl -X PUT --json '{"name":"Ada"}' https://api.example.com/users/42
curl -X PUT --json '{"name":"Ada"}' https://api.example.com/users/42
```

CODE EXPLAINED

→ The client makes the key once and reuses it for every retry

→ Server stores key → (payload hash, response) for a day or so

→ Same key + same payload = replay; different payload = error

→ PUT needs no key: replaying it can't create a duplicate

IN YOUR STACK

- **Express** — No built-in; middleware that stores key → response in Redis
- **Django** — No built-in; unique `idempotency_key` column + `transaction.atomic()`
- **Spring** — No built-in; a `HandlerInterceptor` or filter + unique DB constraint
- **Rails** — No built-in; `create_or_find_by!(key:)` on a unique index
- **ASP.NET** — No built-in; endpoint filter + `IDistributedCache`
- **Go** — Middleware wrapping the handler; key table with a `UNIQUE` index

METHOD PROPERTIES

METHOD	SAFE	IDEMPOTENT	CACHEABLE
GET	✔	✔	✔
HEAD	✔	✔	✔
OPTIONS	✔	✔	✖
PUT	✖	✔	✖
DELETE	✖	✔	✖
POST	✖	✖	rarely
PATCH	✖	✖	✖

WHEN TO RETRY

SITUATION	RETRY?
Network error on GET / PUT / DELETE	Yes, with backoff
Network error on POST	Only with an `Idempotency-Key`
429 · 503	Yes, after `Retry-After`
408 · 502 · 504	Yes, with backoff
Other 4xx	No: fix the request
500	Maybe once; it may repeat

⚠ COMMON MISTAKES

- Retrying POST automatically (client libraries, queues): duplicate orders and double charges
- Making a GET increment a counter or send an email: prefetchers and retries will repeat it
- Generating a new idempotency key on each retry, which defeats the point

✓ TIPS & BEST PRACTICES

- Disable a submit button after click and send an idempotency key; do both
- Use PUT with a client-chosen ID (`PUT /orders/{uuid}`) to make creates naturally idempotent
- Expire stored keys after 24 hours or so; long enough for any retry window

QUICK SUMMARY

- `safe` — read-only (GET, HEAD, OPTIONS)
- `idempotent` — same effect on repeat (+ PUT, DELETE)
- `Idempotency-Key` — makes POST retry-safe
- Retry `429` / `503` after `Retry-After`

Sheet 07

1xx Informational & 2xx Success

100 Continue · 101 Switching Protocols · 103 Early Hints · 200 · 201 · 202 · 204 · 206

IT WORKED, AND HOW

2xx codes tell the client the request succeeded, and the exact code says what kind of success: `200` returned data, `201` created something (with `Location`), `202` accepted work that will finish later, `204` succeeded with nothing to send back, and `206` returned only the byte range asked for. **1xx** codes are interim: the final response follows on the same request. Picking the precise code lets clients react without parsing the body.

NOTES

- Every `2xx` is success; clients should treat unknown `2xx` like `200`
- `204` must not have a body; `200` with `{}` is not the same
- `103 Early Hints` lets the browser preload CSS/JS while the server works
- `101` is how a request upgrades to WebSockets (HTTP/1.1)

SUCCESS VARIATIONS

```
GET /videos/intro.mp4 HTTP/1.1
Range: bytes=0-1048575

HTTP/1.1 206 Partial Content
Content-Range: bytes 0-1048575/73400320
Accept-Ranges: bytes
Content-Length: 1048576

POST /reports HTTP/1.1
Content-Type: application/json

{"type":"sales","year":2026}

HTTP/1.1 202 Accepted
Location: /reports/jobs/77

GET /chat HTTP/1.1
Upgrade: websocket
Connection: Upgrade

HTTP/1.1 101 Switching Protocols
Upgrade: websocket
```

CODE EXPLAINED

→ `Range` asks for the first 1 MiB; `206` + `Content-Range` delivers it

→ `Accept-Ranges: bytes` tells clients they can resume or seek

→ `202` means "queued"; `Location` is where to poll (sheet 23)

→ `101` switches the connection to another protocol

IN YOUR STACK

- **Express** — `res.sendStatus(204)` · `res.status(202).location(url).end()`
- **Django** — `HttpResponse(status=204)` · DRF `Response(status=status.HTTP_202_ACCEPTED)`
- **Spring** — `ResponseEntity.noContent().build()` · `ResponseEntity.accepted().location(uri).build()`
- **Rails** — `head :no_content` · `head :accepted, location: job_url(job)`
- **ASP.NET** — `Results.NoContent()` · `Results.Accepted(url)`
- **Go** — `w.WriteHeader(http.StatusNoContent)` · `http.StatusAccepted`; `http.ServeContent` handles `Range`

1XX & 2XX CODES

CODE	NAME	USE IT WHEN
100	Continue	Client sent `Expect: 100-continue` before a big upload
101	Switching Protocols	WebSocket upgrade
103	Early Hints	`Link` preload headers before the final response
200	OK	Data returned (GET, or PUT/PATCH with body)
201	Created	New resource; include `Location`
202	Accepted	Work queued, not done
204	No Content	Done, nothing to return (DELETE, PUT)
206	Partial Content	Answering a `Range` request

WHICH SUCCESS CODE

REQUEST	RETURN
`GET item / list`	`200`
`POST create`	`201` + `Location`
`POST long job`	`202` + status URL
`PUT / PATCH`	`200` + body or `204`
`DELETE`	`204`
`POST action, result`	`200` + result

⚠ COMMON MISTAKES

- Returning `200` with `{"error": ...}` in the body: clients and monitoring think it worked
- Sending a body with `204`; some clients hang or drop it
- Using `201` without a `Location` header

✓ TIPS & BEST PRACTICES

- Return the created object with `201` so the client has server-set fields (ID, timestamps)
- Serve large files through your framework's range-aware helper instead of streaming by hand
- Document every success code an endpoint can return, not just `200`

QUICK SUMMARY

- `200` — OK with data
- `201` — created + `Location`
- `202` — accepted, finishes later
- `204` — success, empty body

Sheet 08

3xx Redirects & Not Modified

301 vs 308 · 302 vs 307 · 303 See Other · Post/Redirect/Get · Location · 304 Not Modified · loops

GO SOMEWHERE ELSE, OR USE YOUR COPY

A **3xx** response tells the client to look elsewhere, with the new URL in `Location`. Two questions pick the code: is the move **permanent** (clients and search engines update links), and must the client **keep the method and body**? `307` and `308` keep them; `301` and `302` let browsers turn POST into GET; `303` always means "now GET this". `304 Not Modified` is different: it answers a conditional request and says the cached copy is still good.

NOTES

- Browsers follow redirects automatically; `curl` only with `-L`
- Browsers cache `301` / `308` hard, sometimes for months
- `Location` may be relative (`/orders/9001`)
- `304` has no body; the client reuses what it cached (sheet 12)

POST/REDIRECT/GET AND A 304

```
POST /orders HTTP/1.1
Content-Type: application/x-www-form-urlencoded

sku=A-100&qty=2

HTTP/1.1 303 See Other
Location: /orders/9001

GET /orders/9001 HTTP/1.1

GET /api/v1/users HTTP/1.1

HTTP/1.1 308 Permanent Redirect
Location: /api/v2/users

GET /logo.png HTTP/1.1
If-None-Match: "a1b2c3"

HTTP/1.1 304 Not Modified
ETag: "a1b2c3"
Cache-Control: max-age=3600
```

CODE EXPLAINED

→ **PRG**: answer a form POST with `303` so refresh doesn't resubmit

→ The browser then GETs the `Location` URL

→ `308` moves an endpoint for good and keeps the method

→ `304` : the ETag still matches, so no body is re-sent

IN YOUR STACK

- **Express** — `res.redirect(303, '/orders/9001')` (default is 302)
- **Django** — `redirect('order-detail', pk=9001)` (302); `permanent=True` for 301
- **Spring** — MVC: `return "redirect:/orders/9001"` ; API: `ResponseEntity.status(303).location(uri).build()`
- **Rails** — `redirect_to order_path(o), status: :see_other` (default 302)
- **ASP.NET** — `Results.Redirect(url)` (302); `permanent: true, preserveMethod: true` for 308
- **Go** — `http.Redirect(w, r, "/orders/9001", http.StatusSeeOther)`

DEBUG IT

- `curl -sIL url` lists every hop and its status
- `ERR_TOO_MANY_REDIRECTS` behind a proxy: app thinks it's on http; trust `X-Forwarded-Proto`
- A stale 301 sticks in the browser cache: test with DevTools "Disable cache" or a private window

REDIRECT CODES

CODE	PERMANENT	KEEPS METHOD	USE
301	✔	✖ (POST→GET)	Old page URLs, SEO moves
308	✔	✔	Moved API endpoints
302	✖	✖ (POST→GET)	Legacy temporary redirect
307	✖	✔	Temporary, keep method + body
303	✖	→ GET	After POST (PRG), job results
304	—	—	Conditional GET: use your cache

REDIRECTS IN APIS

SITUATION	DO
Endpoint renamed	308 for a while, then remove
Trailing slash mismatch	Pick one; 308 the other
http → https for APIs	Reject instead (token already leaked)
Login required (browser)	302/303 to login page
Login required (API)	401, never a redirect

⚠ COMMON MISTAKES

- Using `301` / `302` to move a POST endpoint: browsers switch it to GET and the body is lost
- Redirecting API clients to an HTML login page instead of returning `401`
- Shipping a `301` by mistake: users' browsers keep obeying it after you fix the server

✓ TIPS & BEST PRACTICES

- Start with `307` while testing a move; switch to `308` once you're sure
- Always use PRG (`303`) after HTML form posts
- Keep redirect chains to one hop; each hop is a full round trip

QUICK SUMMARY

- `301` / `308` — permanent (308 keeps method)
- `302` / `307` — temporary (307 keeps method)
- `303` — "now GET this" after POST
- `304` — not modified; use the cache

4xx Client Errors

400 · 401 vs 403 · 404 vs 410 · 405 · 409 · 412 · 415 · 422 · 429

THE CLIENT NEEDS TO CHANGE SOMETHING

A **4xx** code says the request itself is the problem, so sending it again unchanged won't help (except 408 and 429). The most-confused pair is **401** ("who are you?": missing or bad credentials) versus **403** ("I know who you are, and no"). Use **400** for requests you can't parse, **422** for well-formed requests that fail validation, and **409** when the request clashes with the current state.

NOTES

- `401` must send `WWW-Authenticate` saying how to authenticate
- `405` must send `Allow` listing the methods that do work
- `429` should send `Retry-After` (sheet 23)
- RFC 9110 renamed 422 to **Unprocessable Content** and 413 to **Content Too Large**

SECURITY CHECK

- Return `404` instead of `403` when even revealing that a resource exists is a leak
- Login failures: one generic message; don't say whether the email exists
- Rate-limit `401` s per account and per IP to slow credential stuffing

COMMON 4XX RESPONSES

```
GET /reports HTTP/1.1

HTTP/1.1 401 Unauthorized
WWW-Authenticate: Bearer realm="api"

DELETE /reports/7 HTTP/1.1
Authorization: Bearer eyJhbGciOi...

HTTP/1.1 403 Forbidden
Content-Type: application/problem+json

{"title":"Only owners can delete reports","status":403}

PUT /users/42 HTTP/1.1
If-Match: "v41"

HTTP/1.1 412 Precondition Failed

POST /users HTTP/1.1
Content-Type: application/json

{"email":"ada@example.com"}

HTTP/1.1 409 Conflict
Content-Type: application/problem+json

{"title":"Email already registered","status":409}
```

CODE EXPLAINED

→ No credentials sent → `401` + a challenge

→ Valid token, wrong role → `403`

→ `If-Match` lost a race with another edit → `412` (sheet 12)

→ Duplicate unique value → `409` ; bad email format would be `422`

IN YOUR STACK

- **Express** — `res.status(404).json(...)` ; validation with `express-validator` or `zod`
- **Django** — DRF raises `NotFound` · `PermissionDenied` · `ValidationError` (400 by default)
- **Spring** — `throw new ResponseStatusException(HttpStatus.NOT_FOUND)` ; `@Valid` failures → 400
- **Rails** — `RecordNotFound` → 404 automatically; `render json: e, status: :unprocessable_entity`
- **ASP.NET** — `Results.NotFound()` · `Results.Forbid()` · `Results.ValidationProblem(errors)`
- **Go** — `http.NotFound(w, r)` · `http.Error(w, msg, http.StatusConflict)`

DEBUG IT

- `401` while sending a token: check `exp` and `aud` ; the `WWW-Authenticate` error text often says which
- `403` on POST in Django, Rails or Spring: usually a missing CSRF token (sheet 13)
- `413` before your code runs: the proxy's body limit (`client_max_body_size` in nginx)

4XX CODES

CODE	NAME	USE IT WHEN
400	Bad Request	Malformed JSON, missing required param
401	Unauthorized	No / invalid credentials
403	Forbidden	Authenticated, not allowed
404	Not Found	No such resource (or hiding it)
405	Method Not Allowed	Route exists, method doesn't; send `Allow`
409	Conflict	Duplicate, wrong state, version clash
410	Gone	Removed for good
412	Precondition Failed	`If-Match` / `If-Unmodified-Since` failed
413	Content Too Large	Body over the limit
415	Unsupported Media Type	Wrong `Content-Type`
422	Unprocessable Content	Parsed fine, failed validation
429	Too Many Requests	Rate limited; send `Retry-After`

400 OR 422?

PROBLEM	CODE
`Body isn't valid JSON`	400
`Wrong type ("abc" for a number)`	400 or 422
`Email format wrong`	422
`Required field missing`	422 (or 400)
`Date range ends before it starts`	422

401 OR 403?

SITUATION	CODE
`No token`	401
`Expired / bad token`	401
`Valid token, missing scope`	403
`Not the owner`	403 or 404

⚠ COMMON MISTAKES

- Using `401` for "not allowed": clients will log the user out and retry login
- Returning `500` for bad input because validation threw an unhandled exception
- Mixing 400 and 422 at random; pick a rule and use it everywhere

✓ TIPS & BEST PRACTICES

- Return every validation error at once, not just the first (sheet 10)
- Send `Allow` with `405` and `Retry-After` with `429` ; clients use them
- Map your framework's not-found and validation exceptions to codes in one place

QUICK SUMMARY

- `401` — who are you? · `403` — not you
- `400` — can't parse · `422` — invalid values
- `404` · `409` · `412` — missing · conflict · stale
- `429` — slow down, see `Retry-After`

Sheet 10

5xx Server Errors & Error Bodies

500 · 502 vs 504 · 503 + Retry-After · application/problem+json · type · title · status · detail

OUR FAULT, SAID CLEARLY

A **5xx** code means the server failed at a request that may well have been fine. `500` is the catch-all for bugs; `502` and `504` come from a **proxy or gateway** that couldn't get a good answer from the app behind it; `503` means "down on purpose or overloaded, try later". Whatever the code, give errors one consistent body shape. **Problem Details** (RFC 9457, `application/problem+json`) is the standard one.

NOTES

- `type` is a URI naming the error kind; `about:blank` means "just the status"
- Add your own fields (`errors` , `traceId`) next to the standard ones
- `503` + `Retry-After` is the right answer during maintenance
- Same shape for 4xx and 5xx, so clients parse errors one way

SECURITY CHECK

- Never return stack traces, SQL or file paths: turn debug mode off in production
- Log details server-side with a request ID and return only the ID
- Don't echo raw user input in `detail` ; it can carry script or log-injection text

PROBLEM DETAILS BODIES

```
HTTP/1.1 422 Unprocessable Content
Content-Type: application/problem+json

{
  "type": "https://api.example.com/problems/validation",
  "title": "Your request has invalid fields",
  "status": 422,
  "detail": "2 fields failed validation",
  "instance": "/orders",
  "errors": [
    {"pointer": "/email", "detail": "must be a valid email"},
    {"pointer": "/quantity", "detail": "must be at least 1"}
  ]
}

HTTP/1.1 500 Internal Server Error
Content-Type: application/problem+json

{"type":"about:blank","title":"Internal Server Error",
 "status":500,"traceId":"7f3a9c21e4b0"}

HTTP/1.1 503 Service Unavailable
Retry-After: 120
```

CODE EXPLAINED

- → `title` is short and stable; `detail` is specific to this request
- → `errors` is an extension: one entry per bad field, as JSON Pointers
- → The 500 body gives a `traceId` to quote to support, nothing else
- → `Retry-After: 120` = try again in two minutes

IN YOUR STACK

- **Express** — No built-in; a last error middleware `(err, req, res, next)` that writes the problem body
- **Django** — DRF `EXCEPTION_HANDLER` setting reshapes errors; `DEBUG = False` in production
- **Spring** — `ProblemDetail` + `spring.mvc.problemdetails.enabled=true` ; `@ControllerAdvice` for custom types
- **Rails** — `rescue_from` in `ApplicationController` ; `consider_all_requests_local = false` in production
- **ASP.NET** — `builder.Services.AddProblemDetails()` + `app.UseExceptionHandler()`
- **Go** — Recovery middleware + `w.Header().Set("Content-Type", "application/problem+json")`

DEBUG IT

- `502` : the proxy reached nothing useful; the app crashed, isn't listening, or closed the connection
- `504` : the app is up but slower than the proxy's timeout; find the slow query
- Match the `traceId` / `X-Request-Id` in proxy and app logs to follow one request

5XX CODES

CODE	NAME	MEANS
500	Internal Server Error	Unhandled bug
501	Not Implemented	Method not supported at all
502	Bad Gateway	Proxy got a bad / no response
503	Service Unavailable	Overloaded or maintenance; `Retry-After`
504	Gateway Timeout	Proxy gave up waiting

PROBLEM DETAILS FIELDS

FIELD	HOLDS
type	URI for the error kind (docs page)
title	Short summary, same for every instance
status	The HTTP status, repeated
detail	What went wrong this time
instance	URI for this occurrence
(extensions)	`errors`, `traceId`, `balance`…

⚠ COMMON MISTAKES

- Returning HTML error pages from a JSON API: clients choke on `<!DOCTYPE`
- Using `500` for validation failures; clients and alerts can't tell bugs from bad input
- A different error shape per endpoint, so clients need custom parsing for each

✓ TIPS & BEST PRACTICES

- Point `type` at a docs page that explains how to fix the problem
- Return a request ID header on every response, not just errors
- Alert on `5xx` rate; don't alert on `4xx` spikes alone

QUICK SUMMARY

- `500` — our bug · `503` — try later
- `502` / `504` — proxy got bad / no answer
- `application/problem+json` — standard error body
- `traceId` — link the client's error to your logs

Core Headers & Content Negotiation

METADATA THAT SHAPES EVERY EXCHANGE

Headers describe the message: `Content-Type` says what the body **is**, `Accept` says what the client **wants back**. Matching them is **content negotiation**: the client lists media types, languages and encodings it can handle, and the server picks one and says so. When nothing fits, the server answers `406 Not Acceptable`; when it can't read what was sent, `415 Unsupported Media Type`.

NOTES

- `q=` weights rank choices: `Accept: application/json, text/html;q=0.8`
- Set `Vary` to the request headers that changed the response (sheet 12)
- Custom headers no longer need an `X-` prefix (RFC 6648)
- Most JSON APIs ignore `Accept` and always return JSON; that's fine if documented

SECURITY CHECK

- Send `X-Content-Type-Options: nosniff` so browsers trust your `Content-Type`
- Reject request bodies whose `Content-Type` you don't expect; cap body size
- Never serve user uploads as `text/html`; use `Content-Disposition: attachment`

NEGOTIATING FORMAT AND ENCODING

```
GET /reports/q3 HTTP/1.1
Accept: application/json, text/csv;q=0.5
Accept-Encoding: gzip, br
Accept-Language: en-US, en;q=0.8

HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Encoding: br
Content-Language: en-US
Vary: Accept, Accept-Encoding

POST /avatars HTTP/1.1
Content-Type: multipart/form-data; boundary=XyZ

--XyZ
Content-Disposition: form-data; name="file"; filename="me.png"
Content-Type: image/png

(binary bytes)
--XyZ--
```

CODE EXPLAINED

- → Client prefers JSON, accepts CSV at lower weight
- → Server compressed with Brotli and says so in `Content-Encoding`
- → `Vary` tells caches which request headers picked this version
- → Multipart: parts separated by the `boundary`; each has its own headers

IN YOUR STACK

- **Express** — `express.json()` · `express.urlencoded()` · `multer` · `compression()`; `req.accepts('json')`
- **Django** — `request.POST` · `request.FILES`; DRF `parser_classes` / `renderer_classes`; `GZipMiddleware`
- **Spring** — `@PostMapping(consumes = ..., produces = ...)`; `MultipartFile`; `server.compression.enabled`
- **Rails** — `respond_to { |f| f.json; f.csv }`; `params` parses JSON and forms; Active Storage uploads
- **ASP.NET** — `[Consumes]` / `[Produces]`; `IFormFile`; `AddResponseCompression()`
- **Go** — `json.NewDecoder(r.Body)` · `r.ParseMultipartForm(32 << 20)` · `http.MaxBytesReader`

DEBUG IT

- Garbled bytes from `curl`: the body is compressed; add `--compressed`
- DevTools shows JSON as plain text: server sent the wrong `Content-Type`
- `415` = your request's `Content-Type`; `406` = your `Accept`

HEADERS YOU'LL SEE EVERYWHERE

HEADER	DIR	PURPOSE
`Host`	req	Which site (required in 1.1)
`User-Agent`	req	Client name / version
`Accept / -Encoding / -Language`	req	What the client can take
`Content-Type`	both	Media type of the body
`Content-Length`	both	Body size in bytes
`Content-Encoding`	both	`gzip · br · zstd`
`Content-Disposition`	res	`attachment; filename="q3.csv"`
`X-Request-Id`	both	Trace one request across services

BODY FORMATS

CONTENT-TYPE	USE
`application/json`	API default
`application/x-www-form-urlencoded`	HTML forms, a=1&b=2
`multipart/form-data`	Forms with files
`application/octet-stream`	Raw binary
`text/csv`	Exports
`application/x-ndjson`	One JSON object per line (streams)

⚠ COMMON MISTAKES

- Sending JSON without `Content-Type: application/json`; many parsers then ignore the body
- Compressing already-compressed files (PNG, ZIP): CPU cost, no gain
- Setting `Content-Length` by hand and getting it wrong; let the framework do it

✓ TIPS & BEST PRACTICES

- Always add `charset=utf-8` to text types or rely on JSON's UTF-8 default
- Upload big files straight to object storage with a pre-signed URL
- Log `Content-Type` and `Content-Length` on 400/415 to debug client bugs fast

QUICK SUMMARY

- `Content-Type` — what I'm sending
- `Accept` — what I want back
- `Content-Encoding` — how it's compressed
- `406` / `415` — can't produce / can't read

Caching Headers

DON'T SEND WHAT THEY ALREADY HAVE

`Cache-Control` tells browsers and shared caches (CDNs, proxies) **whether** they may store a response and **how long** it stays fresh. When it goes stale, the client **revalidates** with a conditional request: `If-None-Match` with the `ETag` it has, or `If-Modified-Since` with `Last-Modified`. If nothing changed, the server sends `304 Not Modified` with no body. The same validators guard writes: `If-Match` makes an update fail with `412` if someone else changed it first.

NOTES

- `no-cache` = store it, but revalidate every time; `no-store` = don't store at all
- `private` = browser only; `public` / `s-maxage` = CDNs may cache too
- A **weak** ETag (`W/"v42"`) means "same meaning", not byte-identical
- `Age` on a response = seconds it has sat in a shared cache

SECURITY CHECK

- Personal or authenticated responses: `Cache-Control: private, no-store`
- A `public` response that varies by user leaks one user's data to the next
- Logout pages and token endpoints must be `no-store`

FRESH, REVALIDATED, NEVER STORED

```
GET /api/products/7 HTTP/1.1

HTTP/1.1 200 OK
Cache-Control: public, max-age=60
ETag: "v42"
Last-Modified: Mon, 02 Nov 2026 15:00:00 GMT
Vary: Accept-Encoding

GET /api/products/7 HTTP/1.1
If-None-Match: "v42"

HTTP/1.1 304 Not Modified
ETag: "v42"
Cache-Control: public, max-age=60

GET /api/me HTTP/1.1
Authorization: Bearer eyJhbGciOi...

HTTP/1.1 200 OK
Cache-Control: private, no-store
```

CODE EXPLAINED

→ Fresh for 60 s in the browser and in shared caches (`public`)

→ After 60 s the client asks "still `v42` ?" and gets an empty `304`

→ `/api/me` is per-user, so nothing may store it

→ Hash the content or use a version column to make ETags

IN YOUR STACK

- **Express** — ETags on by default (`app.set('etag')`); `res.set('Cache-Control', 'no-store')` ; `express.static(dir, { maxAge: '1y' })`
- **Django** — `@cache_control(max_age=60)` · `@never_cache` · `ConditionalGetMiddleware`
- **Spring** — `ResponseEntity.ok().cacheControl(CacheControl.maxAge(Duration.ofSeconds(60))).eTag("v42")`
- **Rails** — `expires_in 1.minute, public: true` · `fresh_when(@product)` · `stale?`
- **ASP.NET** — `[ResponseCache(Duration = 60)]` for headers; `AddOutputCache()` for a server cache
- **Go** — `w.Header().Set("Cache-Control", ...)` ; `http.ServeContent` answers conditional requests

DEBUG IT

- DevTools **Size** column: `(memory cache)` / `(disk cache)` = never hit the network
- CDN hit or miss: look for `Age` , `X-Cache: HIT` or `CF-Cache-Status`
- Normal reload revalidates; **Empty Cache and Hard Reload** refetches everything

CACHE-CONTROL DIRECTIVES

DIRECTIVE	MEANS
`max-age=N`	Fresh for N seconds
`s-maxage=N`	Override for shared caches
`no-cache`	Revalidate before every use
`no-store`	Never store
`private / public`	Browser only / any cache
`must-revalidate`	No stale use after expiry
`immutable`	Never changes (hashed asset URLs)
`stale-while-revalidate=N`	Serve stale while refreshing

VALIDATORS & CONDITIONAL REQUESTS

HEADER	ROLE
`ETag`	Version tag for the representation
`If-None-Match`	GET: send body only if changed → `304`
`If-Match`	PUT/PATCH: only if unchanged → else `412`
`Last-Modified`	Timestamp validator (1 s resolution)
`If-Modified-Since`	Date-based conditional GET
`Vary`	Request headers that change the response

⚠ COMMON MISTAKES

- Thinking `no-cache` means "don't cache"; that's `no-store`
- Long `max-age` on URLs that change (e.g. `/app.js`); use hashed file names instead
- Forgetting `Vary: Accept-Encoding` or `Vary: Origin` when the response depends on them

✓ TIPS & BEST PRACTICES

- Static assets: hashed names + `max-age=31536000, immutable`
- Use `If-Match` + ETag for optimistic locking on updates
- Pick one: cache at the CDN with `s-maxage` , or not at all; partial setups confuse

QUICK SUMMARY

- `max-age` — how long it's fresh
- `no-store` — never cache; `no-cache` — always revalidate
- `ETag` + `If-None-Match` — `304` when unchanged
- `If-Match` — `412` on a lost update

Cookies & Sessions

Set-Cookie · HttpOnly · Secure · SameSite · Domain / Path · Max-Age · session IDs · CSRF tokens

STATE ON TOP OF A STATELESS PROTOCOL

The server sets a cookie with `Set-Cookie`; the browser sends it back in `Cookie` on every matching request. A **session** cookie holds a random ID that points to data on the server (or an encrypted, signed blob of the data itself). Because browsers attach cookies automatically, even on requests another site triggers, cookie-based auth needs **CSRF** protection: `SameSite` plus a token the attacker's page can't read.

NOTES

- No `Max-Age` / `Expires` = session cookie, gone when the browser closes
- `Max-Age=0` deletes a cookie
- Each cookie ~4 KB max; keep data on the server, the ID in the cookie
- Mobile apps and server-to-server calls usually use tokens, not cookies

SECURITY CHECK

- `HttpOnly` stops script (XSS) from reading the session cookie
- Name it `__Host-sid` (any `__Host-` name) to force `Secure`, `Path=/` and no `Domain`
- Issue a new session ID at login and logout (session fixation)

LOGIN, THEN USE THE SESSION

```
POST /login HTTP/1.1
Content-Type: application/x-www-form-urlencoded

email=ada%40example.com&password=...

HTTP/1.1 303 See Other
Location: /dashboard
Set-Cookie: __Host-sid=8f2c; Path=/; HttpOnly; Secure; SameSite=Lax; Max-Age=1209600
Set-Cookie: csrftoken=Qx9; Path=/; Secure; SameSite=Lax

GET /dashboard HTTP/1.1
Cookie: __Host-sid=8f2c; csrftoken=Qx9

POST /transfer HTTP/1.1
Cookie: __Host-sid=8f2c; csrftoken=Qx9
X-CSRF-Token: Qx9
Content-Type: application/json

{"to":"acct_7","amount":50}
```

CODE EXPLAINED

- → Session cookie lasts 14 days; script can't read it
- → The CSRF cookie is readable on purpose so your JS can copy it
- → Server checks the header matches the cookie (double-submit)
- → Another site can make the browser send cookies, but can't set the header

IN YOUR STACK

- **Express** — `express-session`; `res.cookie(name, v, { httpOnly: true, secure: true, sameSite: 'lax' })`
- **Django** — Sessions built in; `SESSION_COOKIE_SECURE = True`; `{% csrf_token %}` / `CsrfViewMiddleware`
- **Spring** — Spring Security sessions; CSRF on by default; `server.servlet.session.cookie.same-site=lax`
- **Rails** — `session[:user_id]` (encrypted cookie store); `protect_from_forgery`; `cookies.encrypted`
- **ASP.NET** — `AddAuthentication().AddCookie()`; `CookieOptions { HttpOnly = true, Secure = true }`; antiforgery
- **Go** — `http.SetCookie(w, &http.Cookie{Name: "session", HttpOnly: true, Secure: true, SameSite: http.SameSiteLaxMode})`

DEBUG IT

- DevTools → Application → Cookies shows every flag; a yellow warning says why one was blocked
- Cookie not sent: `Secure` on `http://`, `SameSite=None` without `Secure`, or `Domain` / `Path` mismatch
- Cross-origin `fetch` needs `credentials: 'include'` and CORS credentials (sheet 18)

SET-COOKIE ATTRIBUTES

ATTRIBUTE	EFFECT
`HttpOnly`	Hidden from `document.cookie`
`Secure`	Sent over HTTPS only
`SameSite`	Cross-site sending rule (right)
`Domain=example.com`	Also send to subdomains
`Path=/api`	Only for paths under `/api`
`Max-Age=N / Expires`	Lifetime; else browser session
`Partitioned`	Third-party cookie per top-level site (CHIPS)

SAMESITE VALUES

VALUE	SENT ON CROSS-SITE…
`Strict`	Nothing, not even link clicks
`Lax (default)`	Top-level GET navigations only
`None`	Everything; requires `Secure`

WHERE AUTH LIVES

OPTION	FITS
`Session cookie`	Server-rendered apps, same-site SPAs
`Bearer token`	Mobile, CLI, service-to-service

⚠ COMMON MISTAKES

- Storing the user's role or price in a plain cookie; clients can edit it
- Turning off CSRF protection because "it's an API" while still using cookies
- `SameSite=None` everywhere to make a cross-site setup work

✓ TIPS & BEST PRACTICES

- Same-site SPA + API (`app.` and `api.` on one domain): cookies with `SameSite=Lax` are simplest
- Expire idle sessions server-side; don't rely on the cookie lifetime
- Store sessions in Redis or the DB so any server can read them

QUICK SUMMARY

- `Set-Cookie` / `Cookie` — server sets, browser returns
- `HttpOnly` · `Secure` · `SameSite` — always set all three
- `session ID` — random, server-side data
- `CSRF token` — required with cookie auth

Authentication Basics

WHO ARE YOU, AND WHAT MAY YOU DO?

Authentication proves who is calling; **authorization** decides what they may do. HTTP carries credentials in the `Authorization` header as `<scheme> <credentials>`, and a server that wants them answers `401` with `WWW-Authenticate` naming the scheme. **Basic** sends a Base64 username:password; **Bearer** sends a token (opaque or a JWT, sheet 15); **API keys** identify an application rather than a person; **sessions** use cookies (sheet 13).

NOTES

- Base64 is encoding, not encryption: Basic is only safe over HTTPS
- Bearer means "whoever holds this token is the user"; protect it like a password
- API keys often travel in a custom header such as `X-API-Key`
- Check authorization on every request, at the resource, not just at login

SECURITY CHECK

- Store API keys hashed; show the full key once; give it a prefix (`sk_live_`) so leaks are searchable
- Compare secrets in constant time; never log `Authorization` headers
- Scope keys and tokens to the least access needed, and support rotation

CHALLENGE AND CREDENTIALS

```
GET /reports HTTP/1.1

HTTP/1.1 401 Unauthorized
WWW-Authenticate: Bearer realm="api"

GET /reports HTTP/1.1
Authorization: Basic YWRhOnMzY3JldA==

GET /reports HTTP/1.1
Authorization: Bearer eyJhbGciOiJSUzI1NiJ9...

GET /reports HTTP/1.1
X-API-Key: sk_live_51Hx...

HTTP/1.1 200 OK
```

CODE EXPLAINED

→ No credentials → `401` + which scheme to use

→ `YWRhOnMzY3JldA==` is just `ada:s3cret` in Base64

→ Bearer token: from a login or OAuth flow (sheets 16–17)

→ API key identifies the calling app; pair it with per-key limits

IN YOUR STACK

- **Express** — `passport` strategies, or middleware reading `req.get('Authorization')`
- **Django** — `django.contrib.auth` ; DRF `DEFAULT_AUTHENTICATION_CLASSES` + `permission_classes`
- **Spring** — Spring Security `SecurityFilterChain` ; `.httpBasic()` · `.oauth2ResourceServer(o -> o.jwt())`
- **Rails** — `has_secure_password` · `authenticate_or_request_with_http_token` ; Rails 8 auth generator
- **ASP.NET** — `AddAuthentication().AddJwtBearer()` ; `[Authorize]` / `.RequireAuthorization("policy")`
- **Go** — Middleware: `r.BasicAuth()` or parse `Authorization` ; put the user in `r.Context()`

DEBUG IT

- `curl -v` : is the `> Authorization:` line actually there?
- Clients drop `Authorization` when a redirect goes to another host; call the final URL
- Works locally, `401` behind a proxy: the proxy may strip or rename the header

WAYS TO AUTHENTICATE

METHOD	CARRIED IN	GOOD FOR	WATCH OUT
Session cookie	`Cookie`	Browser apps	CSRF
Basic	`Authorization: Basic`	Internal tools, scripts	Password on every call
API key	X-API-Key / Bearer	Server-to-server, partners	Long-lived; rotate
Opaque token	`Authorization: Bearer`	Revocable access	Lookup per request
JWT	`Authorization: Bearer`	Stateless APIs	Hard to revoke
mTLS	TLS client certificate	Service mesh, banks	Cert management

AUTHORIZATION MODELS

MODEL	RULE
RBAC	Role → permissions
Scopes	Token lists what it may do
Ownership	`resource.owner_id == user.id`
ABAC	Rules on user + resource attributes

⚠ COMMON MISTAKES

- Checking "is logged in" but not "owns this record" (BOLA, the #1 API flaw)
- Putting API keys in URLs (`?api_key=`): they land in logs and browser history
- Shipping a secret API key inside a mobile app or SPA bundle

✓ TIPS & BEST PRACTICES

- Centralize authentication in middleware; do authorization near the data
- Return `401` with `WWW-Authenticate` , `403` when the identity is known
- Use an identity provider instead of storing passwords if you can

QUICK SUMMARY

- `authn` — who · `authz` — what they may do
- `Authorization: <scheme> <credentials>`
- `Basic` · `Bearer` · API key · cookie
- `401` + `WWW-Authenticate` — please authenticate

JSON Web Tokens (JWT)

header.payload.signature · Base64URL · iss · sub · aud · exp · HS256 vs RS256 · verify, don't decode

SIGNED CLAIMS YOU CAN CHECK LOCALLY

A **JWT** is three Base64URL parts joined by dots: a **header** (algorithm, key ID), a **payload** of **claims** (who, for which API, until when) and a **signature**. Any API holding the right key can verify the token without calling the issuer, which is why JWTs suit distributed systems. The payload is **signed, not encrypted**: anyone can read it. The trade-off is revocation: a valid JWT works until it expires.

NOTES

- **HS256** = shared secret: issuer and every API hold the same key
- **RS256 / ES256** = private key signs, public key (from a JWKS URL) verifies
- Times are seconds since 1970 (Unix time), in UTC
- Keep access tokens short-lived (5–15 min) and refresh them

SECURITY CHECK

- Pin the allowed algorithms; reject `alg: none` and HS/RS confusion
- Always check `exp`, `iss` and `aud`, not just the signature
- No secrets or personal data in claims; prefer an `HttpOnly` cookie to `localStorage` in browsers

PEEK AT A TOKEN (NOT VERIFICATION)

```
TOKEN=eyJhbGciOiJSUzI1NiIsImtpZCI6ImsxIn0.eyJpc3Mi...SflKxw
echo "$TOKEN" | cut -d. -f2 | tr '_-' '/+' | base64 -d 2>/dev/null | jq .
```

CODE EXPLAINED

- → Base64URL uses `-` and `_`, hence the `tr` before decoding
- → `kid` picks which public key in the issuer's JWKS verifies it
- → `exp` - `iat` = 900 s: a 15-minute token
- → Verify = check signature, then `iss`, `aud`, `exp`, `nbf`

HEADER AND PAYLOAD

```
{ "alg": "RS256", "typ": "JWT", "kid": "k1" }

{
  "iss": "https://auth.example.com",
  "sub": "user_42",
  "aud": "https://api.example.com",
  "iat": 1793631600,
  "exp": 1793632500,
  "scope": "orders:read orders:write"
}
```

IN YOUR STACK

- **Express** — `jose`: `await jwtVerify(token, JWKS, { issuer, audience })`
- **Django** — `djangorestframework-simplejwt`; or `jwt.decode(t, key, algorithms=["RS256"], audience=aud)`
- **Spring** — Resource server starter; `spring.security.oauth2.resourceserver.jwt.issuer-uri`
- **Rails** — `jwt` gem: `JWT.decode(t, key, true, algorithm: "RS256", aud: aud, verify_aud: true)`
- **ASP.NET** — `AddJwtBearer(o => { o.Authority = issuer; o.Audience = aud; })`
- **Go** — `golang-jwt/jwt/v5`: `jwt.Parse(t, keyFn, jwt.WithValidMethods([]string{"RS256"}))`

DEBUG IT

- `401` with "invalid audience" or "issuer": compare `aud` / `iss` with your config exactly
- Fails only on some servers: clock skew; allow 30–60 s leeway
- "kid not found": the issuer rotated keys; refresh the cached JWKS

REGISTERED CLAIMS

CLAIM	MEANING
`iss`	Issuer (who signed it)
`sub`	Subject (user or client ID)
`aud`	Audience (which API)
`exp`	Expires at
`nbf`	Not valid before
`iat`	Issued at
`jti`	Unique token ID (deny-lists)

ALGORITHMS

ALG	TYPE	USE
`HS256`	HMAC + shared secret	One service issues and verifies
`RS256`	RSA key pair	Most identity providers
`ES256`	ECDSA key pair	Smaller tokens
`EdDSA`	Ed25519	Modern, fast
`none`	Unsigned	Always reject

⚠ COMMON MISTAKES

- Decoding the payload and trusting it without verifying the signature
- Long-lived JWTs (days) with no way to revoke them
- Using JWTs as server sessions just because they're popular; a session cookie is often simpler

✓ TIPS & BEST PRACTICES

- Use a maintained library; never hand-roll signature checks
- Need revocation? Short `exp` + refresh tokens, or a `jti` deny-list
- Decode tokens locally when debugging; don't paste production tokens into websites

QUICK SUMMARY

- `header.payload.signature` — Base64URL, signed
- `iss` · `sub` · `aud` · `exp` — check all four
- `RS256` — sign private, verify public (JWKS)
- Readable by anyone; valid until `exp`

OAuth 2.0 Concepts

DELEGATED ACCESS WITHOUT SHARING PASSWORDS

OAuth 2.0 lets an app (the **client**) get limited access to an API (the **resource server**) on behalf of a user (the **resource owner**), without ever seeing the user's password. An **authorization server** checks who the user is, asks for consent, and issues an **access token** limited by **scopes**. A **grant type** is the recipe for getting that token; machines with no user use **client credentials**.

NOTES

- OAuth is **authorization**; for login use OpenID Connect on top (sheet 17)
- **Confidential** clients (servers) can keep a secret; **public** clients (SPAs, mobile) can't
- Access tokens: short-lived. **Refresh tokens**: long-lived, used to get new access tokens
- Discovery: `/.well-known/openid-configuration` lists every endpoint

SECURITY CHECK

- Never put a client secret in an SPA or mobile app; use PKCE instead (sheet 17)
- Request the smallest scopes; register exact redirect URIs, no wildcards
- Rotate refresh tokens on every use and revoke the family if one is reused

CLIENT CREDENTIALS (MACHINE TO MACHINE)

```
curl -u "$CLIENT_ID:$CLIENT_SECRET" \
     -d grant_type=client_credentials \
     -d scope="reports:read" \
     https://auth.example.com/oauth/token

curl -H "Authorization: Bearer $ACCESS_TOKEN" \
     https://api.example.com/reports

curl -d grant_type=refresh_token \
     -d refresh_token="$REFRESH_TOKEN" \
     -d client_id="$CLIENT_ID" \
     https://auth.example.com/oauth/token
```

CODE EXPLAINED

→ The client authenticates itself with Basic auth at the token endpoint

→ `expires_in: 900` → get a new token after 15 minutes

→ The API only sees the access token and checks its scopes

→ Refresh tokens come from user flows, not client credentials

TOKEN RESPONSE

```
{
  "access_token": "eyJhbGciOiJSUzI1NiIs...",
  "token_type": "Bearer",
  "expires_in": 900,
  "scope": "reports:read"
}
```

IN YOUR STACK

- **Express** — Client: `openid-client`; API: `express-oauth2-jwt-bearer` or `jose`
- **Django** — Server: `django-oauth-toolkit`; client: `authlib`
- **Spring** — `spring-boot-starter-oauth2-client` / `-resource-server`; Spring Authorization Server
- **Rails** — Client: `omniauth-oauth2`; server: `doorkeeper`
- **ASP.NET** — `AddOpenIdConnect()` client · `AddJwtBearer()` API · OpenIddict / Duende server
- **Go** — `golang.org/x/oauth2` and `oauth2/clientcredentials`: `cfg.Token(ctx)`

THE FOUR ROLES

ROLE	EXAMPLE
Resource owner	Ada, the user
Client	The photo-printing app
Authorization server	Google / Okta / Entra ID / Keycloak
Resource server	The Photos API

GRANT TYPES

GRANT	USE	STATUS
`authorization_code + PKCE`	Any app with a user	Default
`client_credentials`	Service to service	Current
`refresh_token`	Renew an access token	Current
`device_code`	TVs, CLIs without a browser	Current
`implicit`	Old SPAs	Removed in OAuth 2.1
`password`	App collects the password	Removed in OAuth 2.1

⚠ COMMON MISTAKES

- Using OAuth access tokens as proof of login ("OAuth for authentication")
- Asking for every scope up front; users decline and admins block the app
- Caching a token past `expires_in` and retrying with it forever

✓ TIPS & BEST PRACTICES

- Refresh a little before expiry, not after a `401`
- Name scopes `resource:action` (`orders:read`) and check them in the API
- Use a hosted identity provider unless running one is your product

QUICK SUMMARY

- `client` gets a token for a `resource server`
- `scope` — what the token may do
- `access` (short) vs `refresh` (long) tokens
- `client_credentials` — no user involved

Sheet 17

OAuth Flows: Authorization Code + PKCE & OpenID Connect

/authorize · state · code_verifier / code_challenge · /token · refresh rotation · id_token · userinfo

THE ONE FLOW EVERY APP SHOULD USE

In the **authorization code** flow the app sends the browser to the authorization server, the user logs in there, and the server redirects back with a one-time **code** that the app swaps for tokens over a back channel. **PKCE** binds the two halves: the app sends a hash of a random **verifier** first and proves it holds the verifier at the swap, so a stolen code is useless. Add the `openid` scope and you get **OpenID Connect**: an **ID token** (a JWT about the user) that makes this a login.

NOTES

- Codes are single-use and expire in about a minute
- `state` ties the callback to the browser that started the flow
- ID token is for the **app** (who logged in); access token is for the **API**
- Implicit flow is gone: SPAs use code + PKCE or a backend-for-frontend

SECURITY CHECK

- Register exact redirect URIs; never accept one from the request without matching
- Validate the ID token: signature, `iss`, `aud` = your client ID, `exp`, `nonce`
- SPAs: a backend-for-frontend (BFF) keeps tokens server-side in an `HttpOnly` session

AUTHORIZATION CODE + PKCE, STEP BY STEP

```
# 1. App makes a random verifier and its SHA-256 challenge
VERIFIER=$(openssl rand -base64 48 | tr -d '=+/' | cut -c1-64)
CHALLENGE=$(printf %s "$VERIFIER" | openssl dgst -sha256 -binary \
            | base64 | tr '+/' '-_' | tr -d '=')

# 2. Browser is sent to the authorization server
https://auth.example.com/authorize?response_type=code
  &client_id=photo-app&redirect_uri=https://app.example.com/callback
  &scope=openid%20profile%20photos:read&state=af0ifjsldkj&nonce=n-0S6
  &code_challenge=$CHALLENGE&code_challenge_method=S256

# 3. User logs in and consents; browser comes back with a code
https://app.example.com/callback?code=SplxlOBeZQQYbYS6&state=af0ifjsldkj

# 4. App swaps code + verifier for tokens (back channel)
curl -d grant_type=authorization_code -d code=SplxlOBeZQQYbYS6 \
     -d redirect_uri=https://app.example.com/callback \
     -d client_id=photo-app -d code_verifier="$VERIFIER" \
     https://auth.example.com/oauth/token
# {"access_token":"...","id_token":"eyJ...","refresh_token":"...","expires_in":900}
```

CODE EXPLAINED

→ Only the **challenge** travels through the browser; the verifier stays in the app

→ The app checks `state` on the callback matches what it sent

→ Confidential clients also send their secret in step 4

→ `nonce` is copied into the ID token; compare it to stop replay

IN YOUR STACK

- **Express** — `openid-client`: `buildAuthorizationUrl()`, then `authorizationCodeGrant()` with the verifier
- **Django** — `authlib`: `oauth.register(...)` · `authorize_redirect()` · `authorize_access_token()`
- **Spring** — `.oauth2Login()` + `spring.security.oauth2.client.registration.*` (PKCE automatic)
- **Rails** — `omniauth_openid_connect` with `pkce: true`
- **ASP.NET** — `AddOpenIdConnect(o => { o.ResponseType = "code"; o.UsePkce = true; })`
- **Go** — `oauth2.GenerateVerifier()` · `AuthCodeURL(state, oauth2.S256ChallengeOption(v))` · `Exchange(ctx, code, oauth2.VerifierOption(v))`

DEBUG IT

- `redirect_uri_mismatch`: must match the registered URI exactly, trailing slash included
- `invalid_grant`: code already used or expired, or the verifier doesn't match the challenge
- Errors come back on the callback as `?error=...&error_description=...`; read the Network tab

WHICH FLOW FOR WHICH APP

APP	FLOW
Server-rendered web app	Code + PKCE + client secret
Single-page app	Code + PKCE, or BFF (preferred)
Mobile / desktop	Code + PKCE via the system browser
CLI / TV	Device code
Service, no user	Client credentials (sheet 16)

WHAT OPENID CONNECT ADDS

PIECE	WHAT IT IS
`openid` scope	Turns OAuth into a login
ID token	JWT: `sub`, `email`, `name`, `nonce`
`/userinfo`	Profile claims for an access token
Discovery	`/.well-known/openid-configuration`
JWKS	Public keys to verify tokens
`end_session`	Log out at the provider

⚠ COMMON MISTAKES

- Sending the ID token to your API as an access token; its `aud` is the client, not the API
- Skipping `state` / `nonce` checks because "PKCE covers it"
- Embedded web views for login on mobile: users can't trust them and providers block them

✓ TIPS & BEST PRACTICES

- Use a certified OIDC client library; the flow has many small checks
- Rotate refresh tokens and store them only on a server or in OS secure storage
- Read endpoints from discovery instead of hard-coding them

QUICK SUMMARY

- `/authorize` → `code` → `/token`
- `PKCE` — verifier + S256 challenge
- `state` · `nonce` — tie callback and token to this login
- `id_token` — who logged in (OIDC)

CORS

same-origin policy · Origin · preflight OPTIONS · Access-Control-Allow-* · credentials + wildcard · Max-Age

LETTING OTHER ORIGINS READ YOUR API

Browsers block a page's script from reading responses from a different **origin** (scheme + host + port). **CORS** is how a server opts in: it answers with `Access-Control-Allow-Origin` and friends. For anything beyond a "simple" request the browser first sends a **preflight** `OPTIONS` asking which method and headers are allowed, and only sends the real request if the answer says yes. CORS is enforced by the **browser**: `curl`, servers and mobile apps ignore it.

NOTES

- `https://app.example.com` and `https://api.example.com` are different origins
- Simple = GET / HEAD / POST, standard headers, form or `text/plain` body; JSON triggers a preflight
- Browsers cap `Access-Control-Max-Age` (Chrome: 2 hours)
- Read non-standard response headers only if listed in `Access-Control-Expose-Headers`

SECURITY CHECK

- Allow-list exact origins; never reflect any `Origin` back with credentials allowed
- `*` is fine for public, cookie-free data and is rejected with credentials anyway
- CORS doesn't protect the API from non-browsers: still authenticate every request

PREFLIGHT, THEN THE REAL REQUEST

```
OPTIONS /api/orders HTTP/1.1
Origin: https://app.example.com
Access-Control-Request-Method: POST
Access-Control-Request-Headers: content-type, authorization

HTTP/1.1 204 No Content
Access-Control-Allow-Origin: https://app.example.com
Access-Control-Allow-Methods: GET, POST, PATCH, DELETE
Access-Control-Allow-Headers: Content-Type, Authorization
Access-Control-Allow-Credentials: true
Access-Control-Max-Age: 600
Vary: Origin

POST /api/orders HTTP/1.1
Origin: https://app.example.com
Content-Type: application/json

HTTP/1.1 201 Created
Access-Control-Allow-Origin: https://app.example.com
Access-Control-Allow-Credentials: true
Access-Control-Expose-Headers: Location
Location: /api/orders/9001
Vary: Origin
```

CODE EXPLAINED

- → Browser asks: may this origin POST with these headers?
- → Server says yes and lets the browser cache that for 10 minutes
- → The real response repeats `Allow-Origin` or script can't read it
- → `Vary: Origin` stops a CDN serving one origin's answer to another

IN YOUR STACK

- **Express** — `app.use(cors({ origin: ['https://app.example.com'], credentials: true }))`
- **Django** — `django-cors-headers` : `CORS_ALLOWED_ORIGINS = [...]` · `CORS_ALLOW_CREDENTIALS = True`
- **Spring** — `CorsConfigurationSource` bean + `.cors(withDefaults())` , or `@CrossOrigin(origins = ...)`
- **Rails** — `rack-cors` : `allow { origins "https://app.example.com"; resource "*", credentials: true }`
- **ASP.NET** — `AddCors(o => o.AddPolicy("app", p => p.WithOrigins(...).AllowCredentials()))` + `UseCors("app")`
- **Go** — `rs/cors` : `cors.New(cors.Options{AllowedOrigins: ..., AllowCredentials: true}).Handler(mux)`

DEBUG IT

- "No 'Access-Control-Allow-Origin' header": open the **preflight** in Network (method OPTIONS) and check its status
- Preflight gets `401` : auth middleware runs before CORS; let OPTIONS through without a token
- A real `500` shows up as a CORS error if error responses skip the CORS headers

CORS RESPONSE HEADERS

HEADER	SAYS
`Access-Control-Allow-Origin`	Which origin may read (one, or `*`)
`Access-Control-Allow-Methods`	Methods allowed (preflight)
`Access-Control-Allow-Headers`	Request headers allowed (preflight)
`Access-Control-Allow-Credentials`	`true` = cookies / auth allowed
`Access-Control-Expose-Headers`	Extra response headers script may read
`Access-Control-Max-Age`	Seconds to cache the preflight

SAME ORIGIN AS HTTPS://APP.EXAMPLE.COM?

URL	SAME?
`https://app.example.com/x`	✔
`http://app.example.com`	✖ scheme
`https://app.example.com:8443`	✖ port
`https://api.example.com`	✖ host

IN DEVELOPMENT

OPTION	NOTE
`Dev-server proxy`	Vite / Next proxy `/api`: no CORS needed
`Allow localhost:5173`	Dev config only

⚠ COMMON MISTAKES

- `Access-Control-Allow-Origin: *` with `Allow-Credentials: true` : browsers reject it
- Trying to fix CORS in the front-end code; only the server's headers matter
- Listing several origins in one `Allow-Origin` header; send the one that matched

✓ TIPS & BEST PRACTICES

- Same-origin is easiest: serve the SPA and `/api` from one host or proxy
- Let a well-tested CORS middleware answer preflights; don't hand-write them
- Test with `curl -i -X OPTIONS -H 'Origin: ...' -H 'Access-Control-Request-Method: POST' url`

QUICK SUMMARY

- `origin` — scheme + host + port
- `preflight` — OPTIONS before non-simple requests
- `Allow-Origin` — exact origin, with `Vary: Origin`
- Browser-only; not a security boundary for the API

REST Principles

MODEL THINGS, NOT FUNCTION CALLS

REST is an architectural style, not a standard: you expose **resources** (orders, customers) at URLs, and clients act on them with the standard HTTP methods, exchanging **representations** (usually JSON). The **uniform interface** means every resource behaves the same way, so clients, caches and tools understand your API without custom rules. Most "REST APIs" stop at level 2 of the Richardson model: resources plus correct methods and status codes.

A RESOURCE WITH LINKS

```
GET /orders/9001 HTTP/1.1
Accept: application/json

HTTP/1.1 200 OK
Content-Type: application/json
ETag: "o9001-v3"

{
  "id": 9001,
  "status": "pending",
  "total": {"amount": 4998, "currency": "USD"},
  "customer": {"id": 42, "href": "/customers/42"},
  "_links": {
    "self":    {"href": "/orders/9001"},
    "items":   {"href": "/orders/9001/items"},
    "cancel":  {"href": "/orders/9001/cancellation", "method": "POST"},
    "payment": {"href": "/orders/9001/payment"}
  }
}
```

REST CONSTRAINTS

CONSTRAINT	IN PRACTICE
Client-server	UI and data evolve separately
Stateless	Every request carries its auth and context
Cacheable	Responses say if they can be cached
Uniform interface	URLs + standard methods + media types + links
Layered system	Proxies, CDNs, gateways are invisible
Code on demand	Optional (e.g. sending JS)

NOTES

- A resource is a noun; a representation is one format of it (JSON, CSV)
- Stateless: no server-side conversation between requests
- **HATEOAS**: responses carry links to what the client can do next
- RPC (`POST /createOrder`) is fine internally; gRPC is RPC done well

CODE EXPLAINED

→ One canonical URL per order; `ETag` makes it cacheable and safely editable

→ Money as integer cents plus currency avoids float errors

→ `_links` (HAL style) shows actions allowed in this state

→ When the order ships, `cancel` simply disappears

IN YOUR STACK

- **Express** — `express.Router()` per resource: `router.route('/:id').get(show).patch(update)`
- **Django** — DRF `DefaultRouter().register('orders', OrderViewSet)` → list + detail routes
- **Spring** — `@RestController` + `@RequestMapping("/orders")` ; Spring HATEOAS `EntityModel` + `linkTo()`
- **Rails** — `resources :orders` → the seven RESTful routes; check with `bin/rails routes`
- **ASP.NET** — `var orders = app.MapGroup("/orders")` then `orders.MapGet("/{id}", ...)`
- **Go** — One pattern per method + path: `"GET /orders/{id}"` , `"PATCH /orders/{id}"`

RICHARDSON MATURITY MODEL

LEVEL	LOOKS LIKE
0	One URL, everything is POST
1	Resources: `/orders/9001`
2	HTTP methods + status codes
3	Hypermedia links (HATEOAS)

STYLE PICKER

STYLE	BEST FOR
REST	Public, cacheable, CRUD-shaped APIs
GraphQL	Varied UI data needs (sheet 25)
gRPC	Fast internal service calls

⚠ COMMON MISTAKES

- Verbs in URLs (`/getOrders` , `/updateUser`) on an API you call REST
- Tunnelling everything through `POST` and returning `200` with error flags
- Exposing database tables one-to-one as resources instead of what clients need

✓ TIPS & BEST PRACTICES

- Design from client use cases, then pick resources
- Level 2 is the practical target; add links where they save clients logic
- Be consistent: one style for naming, errors, paging across the whole API

QUICK SUMMARY

- `resource` — a noun at a URL
- `representation` — JSON (or other) view of it
- `uniform interface` — standard methods + codes
- `HATEOAS` — links to next actions

Resource & URL Design

URLS CLIENTS CAN GUESS

Good URLs are **predictable**: plural nouns for collections, an ID for one item, and HTTP methods for the verbs. Nest only to show ownership one level deep (`/customers/42/orders`), and give every item one **canonical URL** (`/orders/9001`). Actions that don't fit CRUD become a sub-resource (`POST /orders/9001/cancellation`) or a clearly marked custom method. Pick one casing for paths and one for JSON fields, and never mix.

NOTES

- Paths: lowercase, kebab-case (`/order-items`)
- JSON fields: `camelCase` or `snake_case` , the same everywhere
- IDs in URLs are often strings; don't promise they're numbers
- No file extensions (`.json`); use `Accept`

A CONSISTENT URL MAP

```
GET    /customers                  list (paged)
POST   /customers                  create → 201 + Location
GET    /customers/42               read one
PATCH  /customers/42               change some fields
DELETE /customers/42               remove
GET    /customers/42/orders        orders owned by one customer
GET    /orders?customer_id=42      same data, as a filter
GET    /orders/9001                each order has ONE canonical URL
POST   /orders/9001/cancellation   action as a sub-resource
POST   /orders/9001:cancel         action as a custom method (Google AIP)
PUT    /users/42/avatar            singleton sub-resource
GET    /me                         shortcut for the current user
GET    /order-items?order_id=9001  kebab-case paths

{"orderId": 9001, "createdAt": "2026-11-02T15:00:00Z", "isGift": false}
```

CODE EXPLAINED

→ Collection + item URLs cover most endpoints

→ Nesting shows ownership; a filter is the flat alternative

→ `cancellation` is a noun you can create; pick one action style

→ Timestamps: ISO 8601 in UTC with `Z`

IN YOUR STACK

- **Express** — `router.get('/customers/:id/orders', ...)` ; `Router({ mergeParams: true })` for nested routers
- **Django** — `path('customers/<int:pk>/orders/', ...)` ; `drf-nested-routers` for nested viewsets
- **Spring** — `@GetMapping("/customers/{id}/orders")` ; Jackson naming strategy for field casing
- **Rails** — `resources :customers do resources :orders, shallow: true end` ; `member { post :cancel }`
- **ASP.NET** — `group.MapGet("/{id}/orders", ...)` ; System.Text.Json uses camelCase by default
- **Go** — `"GET /customers/{id}/orders"` ; struct tags set casing: `json:"createdAt"`

DO / DON'T

DON'T	DO
`GET /getUsers`	`GET /users`
`POST /users/42/delete`	`DELETE /users/42`
`/user/42`	`/users/42`
`/a/1/b/2/c/3/d`	`/c/3` + filters
`/Users/GetAll.json`	`/users` + `Accept`
`?userId=` and `?user_id=`	One casing everywhere

FIELD CONVENTIONS

DATA	REPRESENT AS
Timestamps	`"2026-11-02T15:00:00Z"`
Money	`{"amount": 4998, "currency": "USD"}`
Enums	Lowercase strings: `"shipped"`
Booleans	`isActive`, `hasMore`
Relations	ID + optional `href`
Unknown / empty	`null` vs omitted: document which

⚠ COMMON MISTAKES

- Deep nesting that forces clients to know every parent ID
- Two URLs for the same item (`/orders/9001` and `/customers/42/orders/9001`) with different behavior
- Exposing auto-increment IDs where guessing them leaks data; use UUIDs or check access

✓ TIPS & BEST PRACTICES

- Write the URL map first and review it before any code
- Prefer filters (`?customer_id=`) over nesting when items have many parents
- Reserve `/me` for the authenticated user's own resources

QUICK SUMMARY

- `/things` + `/things/{id}` — the core pattern
- Methods are the verbs; URLs are nouns
- Nest one level for ownership, else filter
- One casing for paths, one for fields

Sheet 21

Pagination, Filtering & Sorting

limit / offset · cursor pagination · Link header · total counts · filters · ?sort=-created_at · sparse fields

BIG COLLECTIONS IN SMALL PAGES

Never return an unbounded list. **Offset** paging (`page`, `limit`) is simple and lets users jump to page 7, but gets slow on big tables and skips or repeats rows when data changes. **Cursor** (keyset) paging returns an opaque token pointing after the last item; it's fast and stable, at the cost of random access. Filters and sort go in the query string with documented names, and the response says how to get the next page.

NOTES

- Default `limit` (e.g. 25) and a hard maximum (e.g. 100)
- `sort=-created_at,name` : minus means descending
- Totals cost a `COUNT(*)` ; make them optional on large data
- `Link: <...>; rel="next"` (RFC 8288) works for any media type

SECURITY CHECK

- Cap `limit` ; `?limit=1000000` is a denial of service
- Allow-list filter and sort fields; never paste them into SQL
- Make cursors opaque and signed if they encode anything sensitive

CURSOR AND OFFSET RESPONSES

```
GET /orders?status=shipped&created_after=2026-10-01&sort=-created_at&limit=25 HTTP/1.1

HTTP/1.1 200 OK
Content-Type: application/json
Link: </orders?status=shipped&limit=25&cursor=eyJpZCI6OTAwMX0>; rel="next"

{
  "data": [{"id": 9025, "status": "shipped"}, ...],
  "next_cursor": "eyJpZCI6OTAwMX0",
  "has_more": true
}

GET /orders?page=3&per_page=25 HTTP/1.1

HTTP/1.1 200 OK

{"data": [...], "page": 3, "per_page": 25, "total": 1042}

GET /orders/9001?fields=id,status,total HTTP/1.1
```

CODE EXPLAINED

- → Filters and sort ride along in the next link automatically
- → The cursor is Base64 of `{"id":9001}` : the last row seen
- → Server query: `WHERE id < 9001 ORDER BY id DESC LIMIT 26`
- → Fetch one extra row to know `has_more` without counting

IN YOUR STACK

- **Express** — No built-in; validate `req.query` with `zod` ; Prisma `take` + `cursor` + `skip: 1`
- **Django** — DRF `CursorPagination` / `LimitOffsetPagination` ; `django-filter` ; `OrderingFilter`
- **Spring** — `Pageable` : `?page=2&size=25&sort=createdAt,desc` → `Page<T>` or `Slice<T>`
- **Rails** — `pagy` or `kaminari` ; `where(...).order(created_at: :desc).limit(25)`
- **ASP.NET** — `[AsParameters]` query object; EF Core `.Where(o => o.Id < cursor).Take(25)`
- **Go** — `strconv.Atoi(q.Get("limit"))` ; `WHERE id < $1 ORDER BY id DESC LIMIT $2`

OFFSET VS CURSOR

	OFFSET	CURSOR
Jump to page N	✔	✖
Speed on deep pages	Slow (scans skipped rows)	Fast (index seek)
Stable while data changes	✖ skips / repeats	✔
Total count	Easy	Optional, costly
Good for	Admin tables	Feeds, sync, exports

QUERY CONVENTIONS

PARAM	EXAMPLE
Equality	`?status=shipped`
Multiple values	`?status=paid,shipped`
Ranges	`?created_after=2026-10-01`
Search	`?q=lamp`
Sort	`?sort=-created_at,name`
Fields	`?fields=id,status`
Page size	`?limit=25` (max 100)

⚠ COMMON MISTAKES

- Sorting a cursor page by a non-unique column; add the ID as a tie-breaker
- Letting clients pass raw column names straight into `ORDER BY`
- Changing default sort order between releases; clients depend on it

✓ TIPS & BEST PRACTICES

- Index the columns you filter and sort on, in the same order
- Return the same envelope (`data` , `next_cursor`) on every list endpoint
- For exports, cursor paging or an async job (sheet 23), not page 400 of 400

QUICK SUMMARY

- `limit` + `cursor` — fast, stable paging
- `page` + `per_page` — simple, jumpable
- `?field=value` filters · `?sort=-field`
- Always cap `limit`

Versioning & Evolving APIs

CHANGE THE API WITHOUT BREAKING CLIENTS

Most changes don't need a new version: adding endpoints, optional parameters or response fields is **non-breaking** as long as clients ignore fields they don't know. A **breaking** change (removing or renaming a field, changing a type, tightening validation) needs a new version that runs **alongside** the old one for a while. Choose one versioning scheme and announce retirement with the `Deprecation` and `Sunset` headers.

NOTES

- **Tolerant reader**: clients ignore unknown fields and enum values
- Path versions (`/v1`) are the most visible and easiest to route
- Date-based versions (`2026-11-01` , Stripe style) pin each client
- Version the whole API, not each endpoint separately

THREE WAYS TO ASK FOR A VERSION

```
GET /v1/orders/9001 HTTP/1.1

GET /orders/9001 HTTP/1.1
API-Version: 2026-11-01

GET /orders/9001 HTTP/1.1
Accept: application/vnd.example.v2+json

HTTP/1.1 200 OK
Content-Type: application/json
Deprecation: @1798675200
Sunset: Wed, 30 Jun 2027 23:59:59 GMT
Link: <https://api.example.com/docs/migrate-to-v2>; rel="deprecation"
```

CODE EXPLAINED

→ Path, custom header, or vendor media type; pick one

→ `Deprecation` (RFC 9745): deprecated since 31 Dec 2026, as a Unix time

→ `Sunset` (RFC 8594): the date it stops working

→ `Link rel="deprecation"` points to migration docs

IN YOUR STACK

- **Express** — Mount routers: `app.use('/v1', v1)` · `app.use('/v2', v2)`
- **Django** — DRF `URLPathVersioning` or `AcceptHeaderVersioning` ; read `request.version`
- **Spring** — `@RequestMapping("/v1/orders")` ; Spring Framework 7 adds `@GetMapping(version = "2")`
- **Rails** — `namespace :v1 do resources :orders end` → `V1::OrdersController`
- **ASP.NET** — `Asp.Versioning.Http` : `app.NewApiVersionSet()` + `.HasApiVersion(2.0)`
- **Go** — `mux.Handle("/v1/", http.StripPrefix("/v1", v1))`

BREAKING OR NOT?

CHANGE	BREAKING?
Add endpoint / optional param	✖
Add response field	✖ (tolerant readers)
Add enum value	Usually ✖ (warn clients)
Remove or rename field	✔
Change field type / format	✔
Make optional param required	✔
Change default sort / page size	✔ in practice
New error code for old case	✔ in practice

VERSIONING SCHEMES

SCHEME	PROS	CONS
`/v1/ path`	Obvious, cache-friendly	URL changes
`Header`	Clean URLs, fine-grained	Easy to forget
`Media type`	Pure HTTP	Awkward for clients
`Query ?v=2`	Easy to test	Mixes with filters

⚠ COMMON MISTAKES

- Shipping a breaking change in place because "only one client uses that field"
- Bumping the major version for additive changes, multiplying versions to maintain
- Sunsetting without measuring who still calls the old version

✓ TIPS & BEST PRACTICES

- Log the version and client ID on every request so you know who to contact
- Keep a public changelog with dates; link it from the docs
- Add `Sunset` headers months ahead; email owners of active API keys

QUICK SUMMARY

- Additive = safe; remove / rename = new version
- Pick one scheme: path, header or media type
- `Deprecation` + `Sunset` — announce the end
- Tolerant readers ignore what they don't know

Rate Limits, Async Jobs & Webhooks

SLOW DOWN, COME BACK LATER, WE'LL CALL YOU

Rate limits protect the API: over the budget, answer 429 with `Retry-After`, and tell clients their remaining budget in headers. Work that takes longer than a few seconds should return `202 Accepted` with a **status URL** the client polls. Better still, push the result with a **webhook**: an HTTP POST from your server to a URL the client registered, signed so the receiver can trust it.

NOTES

- Many APIs use `X-RateLimit-Limit / -Remaining / -Reset`; `RateLimit` is the IETF draft
- Limit per API key or user, not just per IP
- Webhooks are at-least-once: receivers must handle duplicates
- Receivers should answer `2xx` fast and process in the background

SECURITY CHECK

- Verify the HMAC over the **raw** body with a constant-time compare
- Reject timestamps older than ~5 minutes to stop replays; dedupe by message ID
- Senders: block private IPs and internal hosts as webhook targets (SSRF)

429, A JOB, AND A SIGNED WEBHOOK

```
HTTP/1.1 429 Too Many Requests
Retry-After: 30
RateLimit-Policy: "default";q=100;w=60
RateLimit: "default";r=0;t=30

POST /exports HTTP/1.1

HTTP/1.1 202 Accepted
Location: /exports/jobs/77

GET /exports/jobs/77 HTTP/1.1

HTTP/1.1 200 OK
Retry-After: 5

{"status":"running","progress":0.4}

POST /hooks/payments HTTP/1.1
Content-Type: application/json
Webhook-Id: msg_2b1x
Webhook-Timestamp: 1793631600
Webhook-Signature: v1,K5oZfzN95Z9UVu1EsfQmfVNQhnkZ2pj9o9NDN/H/pI4=

{"type":"payment.succeeded","data":{"id":"pay_77"}}
```

CODE EXPLAINED

→ 100 requests per 60 s; none left; resets in 30 s

→ Poll the job; `Retry-After` hints how often

→ When done the job returns a `303` to the file, or a `result` link

→ Signature = HMAC-SHA256 of `id.timestamp.body` (Standard Webhooks)

IN YOUR STACK

- **Express** — `express-rate-limit`; keep the raw body for signatures: `express.raw({ type: 'application/json' })`
- **Django** — DRF `throttle_classes` (`UserRateThrottle`); Celery jobs; verify `request.body`
- **Spring** — Bucket4j or a gateway; job queue + status table; read the body as `byte[]` to verify
- **Rails** — `rate_limit to: 100, within: 1.minute` (7.2+); Active Job; verify `request.raw_post`
- **ASP.NET** — `AddRateLimiter()` + `.RequireRateLimiting("fixed")`; `BackgroundService` / Hangfire
- **Go** — `x/time/rate` limiter middleware; `io.ReadAll(r.Body)`, then `hmac.Equal`

DEBUG IT

- Watch `RateLimit` / `X-RateLimit-Remaining` across calls to see the budget drain
- Signature mismatch: you verified re-serialized JSON, not the exact bytes received
- Receive webhooks locally with a tunnel (`ngrok`, `stripe listen`, `cloudflared`)

RETRY WITH EXPONENTIAL BACKOFF + JITTER

ATTEMPT	WAIT (BASE 1 S, CAP 60 S)
1	random(0, 1 s)
2	random(0, 2 s)
3	random(0, 4 s)
4	random(0, 8 s)
n	random(0, min(60, 2^{n-1}) s)
any	`Retry-After` wins if present

WEBHOOK CHECKLIST

SIDE	DO
Sender	Sign, include ID + timestamp, retry with backoff for ~3 days
Sender	Event types + a way to replay missed events
Receiver	Verify signature and timestamp first
Receiver	Return 2xx in < 5 s; queue the work
Receiver	Dedupe by `Webhook-Id`

⚠ COMMON MISTAKES

- Retrying in a tight loop on `429`, which keeps you rate-limited
- Doing slow work inside the webhook request, so the sender times out and retries
- Trusting webhook payloads without a signature check

✓ TIPS & BEST PRACTICES

- Add jitter to every retry so clients don't stampede together
- Webhook payloads can be thin (`{type, id}`); the receiver fetches fresh data
- Give each client its own signing secret and support rotating it

QUICK SUMMARY

- `429` + `Retry-After` — wait, then retry
- `202` + `Location` — poll the job
- Webhook — signed POST to the client
- Backoff + jitter for every retry

Documenting with OpenAPI

ONE SPEC, MANY TOOLS

An **OpenAPI** document describes every path, operation, parameter, body and response in YAML or JSON. From it you get interactive docs (Swagger UI, Redoc, Scalar), generated clients and servers, mock servers, request validation and contract tests. Write it **design-first** (spec, then code) or generate it **code-first** from annotations; either way, treat it as part of the API and review changes to it. Version 3.1 uses standard **JSON Schema**.

NOTES

- `operationId` becomes the method name in generated clients
- `$ref` reuses schemas, parameters and responses from `components`
- 3.1 allows `type: [string, "null"]` instead of `nullable`
- Lint specs in CI (Spectral) and diff them for breaking changes

A MINIMAL OPENAPI 3.1 SPEC

```yaml
openapi: 3.1.0
info: { title: Orders API, version: 1.4.0 }
servers: [{ url: https://api.example.com/v1 }]
paths:
  /orders/{id}:
    get:
      operationId: getOrder
      parameters:
        - { name: id, in: path, required: true, schema: { type: integer } }
      responses:
        '200':
          description: The order
          content:
            application/json:
              schema: { $ref: '#/components/schemas/Order' }
        '404': { description: No such order }
      security: [{ bearerAuth: [] }]
components:
  schemas:
    Order:
      type: object
      required: [id, status]
      properties:
        id: { type: integer }
        status: { type: string, enum: [pending, paid, shipped] }
  securitySchemes:
    bearerAuth: { type: http, scheme: bearer, bearerFormat: JWT }
```

CODE EXPLAINED

→ `paths` → URL → method → operation

→ Path params must be `required: true`

→ Responses are keyed by status code, quoted

→ `security` on an operation names a scheme from `securitySchemes`

IN YOUR STACK

- **Express** — `swagger-ui-express` serves a spec; `zod-to-openapi` or `tsoa` generate one
- **Django** — `drf-spectacular`: `SpectacularAPIView` + `SpectacularSwaggerView`
- **Spring** — `springdoc-openapi`: `/v3/api-docs` and `/swagger-ui.html` out of the box
- **Rails** — `rswag` (specs → OpenAPI) or `oas_rails`
- **ASP.NET** — `AddOpenApi()` + `MapOpenApi()` (.NET 9+); Scalar or Swagger UI to view it
- **Go** — `swaggo/swag` from comments, or spec-first with `oapi-codegen`

TOP-LEVEL KEYS

KEY	HOLDS
`openapi`	Spec version (`3.1.0`)
`info`	Title, version, contact, license
`servers`	Base URLs
`paths`	Operations per URL + method
`components`	Reusable schemas, params, responses
`security`	Default auth for all operations
`tags`	Groups operations in the docs
`webhooks`	Outgoing calls you make (3.1)

TOOLING

TOOL	DOES
`Swagger UI / Redoc / Scalar`	Interactive docs
`openapi-generator`	Clients and server stubs, 50+ languages
`Spectral`	Lint style rules
`oasdiff`	Detect breaking changes
`Prism (Stoplight)`	Mock server from the spec
`Schemathesis`	Property-based API tests

⚠ COMMON MISTAKES

- Docs that drift from the code because nobody checks the spec in CI
- Documenting only `200` and leaving out error responses
- Inline schemas everywhere instead of `$ref`, so generated clients get dozens of duplicate types

✓ TIPS & BEST PRACTICES

- Fail CI on spec lint errors and on breaking diffs against `main`
- Add `examples` to schemas; docs and mocks get much better
- Publish the raw spec URL so clients can generate their own SDKs

QUICK SUMMARY

- `paths` — every operation
- `components` + `$ref` — reuse
- `securitySchemes` — how to authenticate
- Spec → docs, clients, mocks, tests

Sheet 25

GraphQL Basics & GraphQL vs REST

single endpoint · POST /graphql · query · mutation · subscription · over- / under-fetching · introspection

ASK FOR EXACTLY THE FIELDS YOU NEED

GraphQL puts one typed **schema** behind a single endpoint. Clients send a **query** that names exactly the fields they want, across related objects, and get back JSON in the same shape: no **over-fetching** (fields you don't need) and no **under-fetching** (five calls to build one screen). Writes are **mutations**; live updates are **subscriptions**. The cost moves to the server: caching, authorization and query cost need more care than in REST.

NOTES

- Usually `POST /graphql` with a JSON body; GET works for queries
- Responses are `200` even when part of the query failed; check `errors` (sheet 28)
- **Introspection** lets tools read the schema (GraphiQL, codegen)
- Same HTTP auth as REST: `Authorization: Bearer ...`

SECURITY CHECK

- Restrict introspection and field suggestions in production for private APIs
- Only accept `application/json` POSTs for mutations, never GET (CSRF)
- First-party apps: allow-list **persisted queries** so random queries can't run

A QUERY OVER HTTP

```
POST /graphql HTTP/1.1
Content-Type: application/json
Authorization: Bearer eyJhbGciOi...

{
  "operationName": "OrderCard",
  "query": "query OrderCard($id: ID!) { order(id: $id) { status total customer { name } } }",
  "variables": { "id": "9001" }
}

HTTP/1.1 200 OK
Content-Type: application/graphql-response+json

{
  "data": {
    "order": { "status": "PENDING", "total": 4998, "customer": { "name": "Ada" } }
  }
}
```

CODE EXPLAINED

- → One request fetches the order and its customer together
- → Only `status`, `total` and the customer's `name` come back
- → `variables` keep values out of the query string
- → `graphql-response+json` is the GraphQL-over-HTTP media type

IN YOUR STACK

- **Express** — Apollo Server (Express integration) or GraphQL Yoga
- **Django** — Strawberry `GraphQLView.as_view(schema=schema)`; Graphene-Django
- **Spring** — Spring for GraphQL: `spring-boot-starter-graphql`, serves `/graphql`
- **Rails** — `graphql-ruby`: `rails g graphql:install` → `GraphqlController#execute`
- **ASP.NET** — Hot Chocolate: `AddGraphQLServer().AddQueryType<Query>()` + `app.MapGraphQL()`
- **Go** — `gqlgen` generates resolvers from the schema; serve with `handler.New(...)`

GRAPHQL VS REST

	REST	GRAPHQL
Endpoints	Many URLs	One (`/graphql`)
Response shape	Server decides	Client decides
Related data	Several calls or `?include=`	One query
HTTP caching	Built in (GET + ETag)	Needs persisted queries / client cache
Errors	Status codes	`errors` array, mostly `200`
Versioning	`/v2`	Add fields, deprecate old ones
Typing	Optional (OpenAPI)	Required (schema)

PICK GRAPHQL WHEN…

SIGNAL	LEAN
Many UIs need different slices of data	GraphQL
Deeply related data on one screen	GraphQL
Public API, simple CRUD	REST
Heavy CDN caching / file downloads	REST
Small team, one client	REST

⚠ COMMON MISTAKES

- Treating GraphQL as a database query language and exposing every table
- Assuming `200 OK` means success; partial errors live in the body
- Skipping pagination on list fields, so one query returns 100,000 rows

✓ TIPS & BEST PRACTICES

- Always name operations (`query OrderCard`); logs and tracing become readable
- Generate typed clients from the schema (GraphQL Code Generator, Apollo iOS / Kotlin)
- Use GraphiQL or Apollo Sandbox to explore the schema

QUICK SUMMARY

- `query` — read · `mutation` — write · `subscription` — live
- One endpoint, client picks the fields
- `200` + `data` / `errors` in the body
- Strong schema, but caching and cost need work

Sheet 26

GraphQL Schema & Types (SDL)

type · scalars · ! non-null · [List] · enum · input · interface / union · Query / Mutation

THE CONTRACT, WRITTEN IN SDL

The **Schema Definition Language** (SDL) declares every type a client can ask for. **Object types** have fields; each field has a type and may take **arguments**. Built-in **scalars** are `ID`, `String`, `Int`, `Float` and `Boolean`; add custom ones like `DateTime`. Fields are **nullable unless marked** `!`. The root types `Query`, `Mutation` and `Subscription` are the entry points; **input** types describe arguments, and **interfaces** and **unions** model "one of several types".

NOTES

- `ID` is serialized as a string, even when it looks numeric
- `Int` is 32-bit; use a custom scalar or `String` for big numbers
- `"Descriptions"` in quotes show up in tools and docs
- `@deprecated(reason: "...")` retires a field without a new version

AN ORDERS SCHEMA

```
  "An order placed by a customer"
type Order implements Node {
  id: ID!
  status: OrderStatus!
  total: Int!                    # cents
  placedAt: DateTime!
  customer: Customer!
  items(first: Int = 20): [OrderItem!]!
  note: String                   # nullable
  legacyRef: String @deprecated(reason: "Use id")
}

interface Node { id: ID! }
enum OrderStatus { PENDING PAID SHIPPED CANCELLED }
scalar DateTime
union SearchResult = Order | Customer | Product

input PlaceOrderInput {
  customerId: ID!
  items: [OrderItemInput!]!
}

type Query {
  order(id: ID!): Order
  orders(status: OrderStatus, first: Int = 25, after: String): OrderConnection!
  search(text: String!): [SearchResult!]!
}

type Mutation {
  placeOrder(input: PlaceOrderInput!): PlaceOrderPayload!
}
```

CODE EXPLAINED

- → `order(id)` returns `Order` (nullable): `null` means not found
- → `[OrderItem!]!` = a list that's never null, of items never null
- → `input` types can only hold scalars, enums and other inputs
- → `orders` returns a connection for cursor paging (sheet 28)

IN YOUR STACK

- **Express** — Schema-first: `typeDefs` in SDL + a `resolvers` map
- **Django** — Code-first: `@strawberry.type class Order: id: strawberry.ID ...` (type hints)
- **Spring** — Schema-first: `src/main/resources/graphql/*.graphqls` + `@QueryMapping`
- **Rails** — Code-first: `class Types::OrderType < Types::BaseObject` + `field :status, ...`
- **ASP.NET** — Code-first: plain C# classes (`Query.GetOrder(id)`) become the schema
- **Go** — Schema-first: `.graphqls` files; `gqlgen generate` makes models and resolver stubs

TYPE KINDS

KIND	EXAMPLE	USE
Scalar	`ID String Int Float Boolean`	Leaf values
Custom scalar	`scalar DateTime`	Validated strings / numbers
Object	`type Order { ... }`	Output data
Enum	`enum OrderStatus { ... }`	Fixed set of values
Input	`input PlaceOrderInput`	Arguments
Interface	`interface Node`	Shared fields
Union	`union SearchResult = A | B`	One of several types

NULLABILITY

TYPE	LIST NULL?	ITEMS NULL?
`[Order]`	✔	✔
`[Order!]`	✔	✖
`[Order]!`	✖	✔
`[Order!]!`	✖	✖

NULL RULE OF THUMB

FIELD	MAKE IT
`Fetched by ID`	Nullable (not found)
`Lists`	`[T!]!`
`Could fail (other service)`	Nullable

⚠ COMMON MISTAKES

- Making everything non-null; one failing field then nulls its whole parent
- Generic `JSON` scalars that throw away the type system
- Separate `createX` / `updateX` arguments instead of one `input` type each

✓ TIPS & BEST PRACTICES

- Design the schema from UI needs, not database tables
- One `input` and one `payload` type per mutation; easy to extend later
- Run a schema diff in CI to catch breaking changes

QUICK SUMMARY

- `type` · `enum` · `input` · `interface` · `union`
- `!` — non-null; nullable by default
- `[T!]!` — safest list type
- `Query` / `Mutation` — entry points

GraphQL Queries, Mutations & Variables

operation names · arguments · $variables · aliases · fragments · @include / @skip · mutation payloads

WRITING OPERATIONS CLIENTS SEND

An **operation** is a named `query`, `mutation` or `subscription` with a **selection set** of fields. Values come in through typed **variables** (`$id: ID!`), sent as a separate JSON object, never pasted into the query text. **Aliases** fetch the same field twice under different names, **fragments** reuse a selection, and the `@include` / `@skip` **directives** turn fields on and off. Mutations return a **payload** you select from like any query.

NOTES

- Variable types must match the argument type exactly (`ID!` ≠ `Int`)
- Default values: `$withItems: Boolean = false`
- Top-level mutation fields run in order; query fields may run in parallel
- `__typename` tells you which type a union / interface returned

QUERY, FRAGMENT, MUTATION

```
query OrderPage($id: ID!, $withItems: Boolean = false) {
  order(id: $id) {
    ...OrderSummary
    items @include(if: $withItems) { sku quantity }
    customer { name email }
  }
  recent: orders(first: 3, status: SHIPPED) {
    edges { node { ...OrderSummary } }
  }
}

fragment OrderSummary on Order { id status total }

mutation PlaceOrder($input: PlaceOrderInput!) {
  placeOrder(input: $input) {
    order { id status }
    errors { field message }
  }
}
```

CODE EXPLAINED

→ `...OrderSummary` pastes the fragment's fields in both places

→ `recent:` is an alias, so `orders` can appear next to `order`

→ `items` only comes back when `$withItems` is true

→ Mutation payload holds the result **and** user-facing errors

IN YOUR STACK

- **Express** — Tests: `server.executeOperation({ query, variables })`
- **Django** — Tests: `schema.execute_sync(query, variable_values={...})`
- **Spring** — Tests: `graphQlTester.document(q).variable("id", "9001").execute()`
- **Rails** — Tests: `MySchema.execute(query, variables: { "id" => "9001" })`
- **ASP.NET** — Tests: `executor.ExecuteAsync(OperationRequestBuilder.New().SetDocument(q)...)`
- **Go** — Tests: `client.New(srv).MustPost(q, &resp, client.Var("id", "9001"))`

VARIABLES (SENT ALONGSIDE)

```
{ "id": "9001", "withItems": true }
{ "input": {
    "customerId": "42",
    "items": [{"sku": "A-100", "quantity": 2}]}}
```

DEBUG IT

- Every call is `POST /graphql` in DevTools: check **Payload** → `operationName` to find yours
- "Cannot query field X on type Y": client and server schemas are out of sync
- "Variable $id of type Int used where ID! expected": fix the variable's declared type

OPERATION FEATURES

FEATURE	SYNTAX
Named operation	`query OrderPage { ... }`
Variable	`($id: ID!)` → `order(id: $id)`
Alias	`recent: orders(first: 3)`
Fragment	`fragment F on Order { id }` → `...F`
Inline fragment	`... on Product { price }`
Directives	`@include(if: $x)` · `@skip(if: $x)`
Type name	`__typename`

MUTATION PAYLOAD PATTERN

FIELD	HOLDS
`order`	The changed object (null on failure)
`errors`	`{ field, message }` validation problems
`clientMutationId`	Optional echo for client matching

CLIENTS

CLIENT	FOR
Apollo Client · urql · Relay	Web / React
Apollo iOS / Kotlin	Mobile

⚠ COMMON MISTAKES

- Building query strings with string interpolation instead of variables (injection, no caching)
- Anonymous operations; you can't tell them apart in logs
- Selecting every field "just in case", bringing over-fetching back

✓ TIPS & BEST PRACTICES

- Co-locate fragments with the UI components that use them
- Return user-fixable errors in the payload; reserve `errors[]` for real failures
- Persist operations at build time and send only their hash

QUICK SUMMARY

- `query Name($v: Type!)` — named, typed operation
- `alias:` · `...Fragment` · `@include` / `@skip`
- Variables go in a separate JSON object
- Mutations return a payload

Sheet 28

GraphQL Resolvers, Errors & Performance

resolver (parent · args · context) · N+1 + DataLoader · errors array · connections · depth / cost limits

RUNNING QUERIES SAFELY AND FAST

Every field has a **resolver**: a function that gets the **parent** object, the field's **arguments** and a per-request **context** (current user, DataLoaders, DB). Resolving a list then a field on each item causes the **N+1 problem**; a **DataLoader** batches those lookups into one query per request. Failures don't change the HTTP status: the field becomes `null` and an entry is added to the top-level `errors` array, with a `path` to the failed field.

NOTES

- Put the authenticated user in `context`; check access in resolvers
- Create DataLoaders per request so caches don't leak between users
- Lists use **connections**: `edges { cursor node }` + `pageInfo`
- `extensions.code` (e.g. `FORBIDDEN`) lets clients branch on error type

SECURITY CHECK

- Limit query depth and cost (fields × list sizes) and set a timeout
- Authorize per field/object; one forgotten resolver leaks data
- Rate-limit by query cost; aliases can pack 100 login attempts into one request

AN N+1 QUERY

```
query {
  orders(first: 50) {
    edges {
      cursor
      node { id customer { name } }   # 50 customer lookups
    }
    pageInfo { hasNextPage endCursor }
  }
}
```

CODE EXPLAINED

→ Without batching: 1 orders query + 50 customer queries

→ DataLoader collects the 50 IDs → `WHERE id IN (...)`, once

→ `invoice` failed, the rest of `order` still came back

→ `path` says exactly which field failed

PARTIAL SUCCESS RESPONSE

```
{
  "data": { "order": { "id": "9001", "invoice": null } },
  "errors": [{
    "message": "Not authorized to view invoice",
    "path": ["order", "invoice"],
    "locations": [{ "line": 3, "column": 5 }],
    "extensions": { "code": "FORBIDDEN" }
  }]
}
```

IN YOUR STACK

- **Express** — `dataloader`: `new DataLoader(ids => getCustomers(ids))` created per request
- **Django** — Strawberry `DataLoader(load_fn=load_customers)` in the context
- **Spring** — `@BatchMapping` on a controller method batches automatically
- **Rails** — `GraphQL::Dataloader`: `dataloader.with(Sources::Record, Customer).load(id)`
- **ASP.NET** — `BatchDataLoader<int, Customer>` or the `[DataLoader]` source generator
- **Go** — `vikstrous/dataloadgen` or `graph-gophers/dataloader`

DEBUG IT

- `200 OK` but the UI shows nothing: read the `errors` array in the response
- Count DB queries per request; 50 identical queries = an N+1
- Turn on resolver tracing (Apollo / OpenTelemetry) to find the slow field

ERROR ENTRY FIELDS

FIELD	HOLDS
`message`	Human-readable text
`path`	Field path that failed
`locations`	Line / column in the query
`extensions`	`code`, retry hints, trace ID

CONNECTION SHAPE

FIELD	HOLDS
`edges { cursor node }`	Items + their cursors
`pageInfo`	`hasNextPage` · `endCursor`
`first / after`	Page size and position

PRODUCTION CHECKLIST

CONTROL	WHY
`Max depth (~10)`	Stops `a { b { a { b ... } } }`
`Cost / complexity limit`	Caps total work per query
`Max first per list`	No 100k-row pages
`Query timeout`	Bounds slow resolvers
`Persisted queries`	Only known operations run
`Introspection off (private)`	Less schema mapping for attackers
`Mask internal errors`	No stack traces in `message`

⚠ COMMON MISTAKES

- A DataLoader shared across requests, leaking one user's data to another
- Throwing raw database errors into `errors[].message`
- Checking auth only at the top-level query while nested fields expose more

✓ TIPS & BEST PRACTICES

- Batch every field that loads by ID; make it the default pattern
- Return domain errors in payloads, unexpected ones in `errors[]`
- Log operation name, cost and duration for every request

QUICK SUMMARY

- `resolver(parent, args, context)` — one per field
- `DataLoader` — batch + cache per request
- `errors[]` + `path` — partial failures
- Limit depth, cost, page size and time

Sheet 29

Quick Reference — Methods & Status Codes

method properties · CRUD → method → status · 2xx · 3xx · 4xx · 5xx · which code when

METHODS (SHEETS 04–06)

METHOD	SAFE	IDEM.	BODY	SUCCESS
GET	✔	✔	✖	200 206 304
HEAD	✔	✔	✖	200 304
OPTIONS	✔	✔	✖	204 + Allow
POST	✖	✖	✔	201 200 202
PUT	✖	✔	✔	200 204 201
PATCH	✖	✖	✔	200 204
DELETE	✖	✔	✖	204 200 202

CRUD MAP (SHEETS 05, 20)

ACTION	REQUEST	SUCCESS	COMMON ERRORS
List	GET /orders	200	400 401
Read	GET /orders/9	200 304	404
Create	POST /orders	201 + Location	409 422
Replace	PUT /orders/9	200 204	404 412 422
Update	PATCH /orders/9	200 204	409 412 422
Delete	DELETE /orders/9	204	404 409
Long job	POST /exports	202 + Location	429

1XX · 2XX · 3XX (SHEETS 07–08)

CODE	NAME	USE
101	Switching Protocols	WebSocket upgrade
200	OK	Data returned
201	Created	+ Location
202	Accepted	Queued, poll later
204	No Content	Done, no body
206	Partial Content	Range answer
301 / 308	Permanent	308 keeps method
302 / 307	Temporary	307 keeps method
303	See Other	GET this after POST
304	Not Modified	Use your cache

4XX (SHEET 09)

CODE	NAME	USE
400	Bad Request	Can't parse
401	Unauthorized	Who are you?
403	Forbidden	Not allowed
404	Not Found	Missing / hidden
405	Method Not Allowed	+ Allow
409	Conflict	State clash
410	Gone	Removed for good
412	Precondition Failed	If-Match lost
413	Content Too Large	Body too big
415	Unsupported Media Type	Bad Content-Type
422	Unprocessable Content	Invalid values
429	Too Many Requests	+ Retry-After

5XX (SHEET 10)

CODE	NAME	MEANS
500	Internal Server Error	Our bug
501	Not Implemented	Method unknown
502	Bad Gateway	Upstream bad answer
503	Service Unavailable	+ Retry-After
504	Gateway Timeout	Upstream too slow

RETRY? (SHEETS 06, 23)

GOT	DO
429 / 503	Wait Retry-After
502 / 504 / network	Backoff + jitter
Other 4xx	Fix the request
POST failed mid-flight	Retry only with Idempotency-Key

WHICH CODE WHEN

SITUATION	CODE
Body isn't valid JSON	400
Valid JSON, bad email	422
No / expired token	401 + WWW-Authenticate
Token OK, wrong role or scope	403
Not the owner, don't reveal it exists	404
Duplicate unique value	409

WHICH CODE WHEN (CONT.)

SITUATION	CODE
Edit lost a race (If-Match)	412
Endpoint moved for good	308
After an HTML form POST	303
Work will finish later	202
Down for maintenance	503 + Retry-After
GraphQL partial failure	200 + errors[] (sheet 28)

Sheet 30

Quick Reference — Headers, Auth & CORS

request / response headers · cache recipes · cookie flags · auth picker · OAuth flow picker · CORS checklist

REQUEST HEADERS (SHEETS 02, 11)

HEADER	EXAMPLE
Host	api.example.com
Accept	application/json
Accept-Encoding	gzip, br
Content-Type	application/json
Authorization	Bearer eyJ...
Cookie	__Host-sid=8f2c
If-None-Match	"v42" → 304
If-Match	"v42" → else 412
Idempotency-Key	UUID per operation
Origin	https://app.example.com
Range	bytes=0-1023

RESPONSE HEADERS (SHEETS 07–13)

HEADER	EXAMPLE
Content-Type	application/json; charset=utf-8
Location	/orders/9001 (201, 3xx, 202)
Cache-Control	private, no-store
ETag	"v42"
Vary	Accept-Encoding, Origin
Set-Cookie	...; HttpOnly; Secure; SameSite=Lax
WWW-Authenticate	Bearer realm="api" (with 401)
Allow	GET, PUT, DELETE (with 405)
Retry-After	30 (with 429, 503)
Link	<...>; rel="next"
Strict-Transport-Security	max-age=31536000

CACHE RECIPES (SHEET 12)

CONTENT	CACHE-CONTROL
Hashed static asset	public, max-age=31536000, immutable
Public API data	public, max-age=60 + ETag
Always fresh, cheap to check	no-cache + ETag
Per-user / authenticated	private, no-store
CDN only	s-maxage=300, max-age=0

COOKIE RECIPE (SHEET 13)

USE	SET-COOKIE
Session	__Host-sid=...; Path=/; HttpOnly; Secure; SameSite=Lax
Delete	same name + Max-Age=0

AUTH PICKER (SHEETS 14–17)

CALLER	USE
Browser, same site	Session cookie + CSRF token
SPA, other domain	BFF, or code + PKCE
Mobile app	Code + PKCE, system browser
Partner server	API key or client credentials
Your services	Client credentials or mTLS
CLI / TV	Device code

JWT CHECKS (SHEET 15)

CHECK	AGAINST
Signature	Pinned alg + JWKS key by kid
iss · aud	Your config, exactly
exp · nbf	Now, ± 60 s leeway

CORS CHECKLIST (SHEET 18)

STEP	HEADER / SETTING
Allow the exact origin	Access-Control-Allow-Origin: https://app.example.com
Cache splits by origin	Vary: Origin
Preflight methods / headers	Allow-Methods · Allow-Headers
Cookies / auth	Allow-Credentials: true, never with *
Expose extra headers	Expose-Headers: Location
Let OPTIONS skip auth	CORS middleware runs first

SECURITY HEADERS

HEADER	VALUE
X-Content-Type-Options	nosniff
Strict-Transport-Security	max-age=31536000; includeSubDomains
Content-Security-Policy	default-src 'self' (HTML)
Referrer-Policy	strict-origin-when-cross-origin
Cache-Control (auth)	no-store